

Résolution numérique des équations de Stokes avec des éléments finis $P^2 - P^1$ sous **MATLAB**

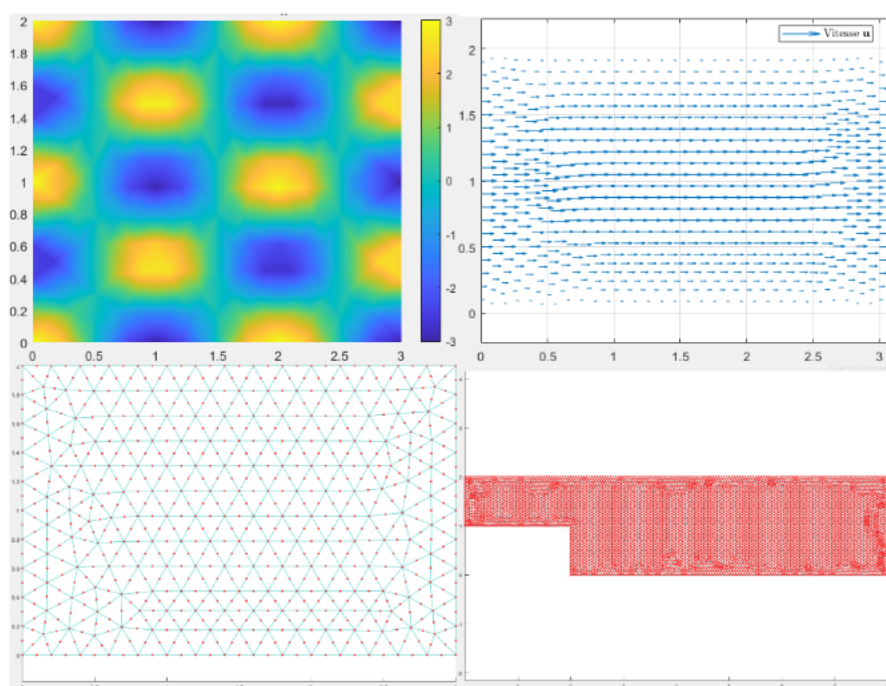


FIGURE 1 – Quelques résultats obtenus lors de ce compte-rendu à titre d'illustration.

Auteurs : Xiaoyu Liu et Benoît Keryer



Novembre 2024

Table des matières

1	Introduction	4
2	Éléments finis P^2 pour le problème de Poisson avec condition de Neumann	5
2.1	Discrétisation	5
2.2	Géométrie et maillage	6
2.2.1	Création des maillages	6
2.2.2	Structures de données du maillage	6
2.2.3	Affichage des maillages	6
2.3	Calcul des matrices élémentaires par passage à l'élément fini de référence	7
2.3.1	Définition des fonctions de base locales	7
2.3.2	Transformation entre $\hat{T}$ et T_ℓ	7
2.3.3	Formulation des intégrales	8
2.3.4	Routine de la matrice de masse élémentaire	10
2.3.5	Routine de la matrice de rigidité élémentaire	11
2.4	Assemblage	13
2.5	Validation	14
3	Éléments finis P2 pour le problème de Poisson avec condition de Dirichlet	19
3.1	Discrétisation	19
3.2	Validation	21
3.2.1	Vérifier les conditions aux bords	21
3.2.2	Calcule f	21
3.2.3	Analyse des erreurs	22
3.3	Conditions de Dirichlet non homogènes	23
3.3.1	Validation	24
4	Équation de Stokes	25
4.1	Maillage	27
4.2	Discrétisation et calcul des matrices élémentaires	27
4.3	Assemblage	30
4.4	Imposition des conditions par pseudo-élimination	31
4.5	Vérification	33
4.5.1	Vérification des équations dans le domaine $\Omega =]0, 3[\times]0, 2[$	34
4.5.2	Vérification des conditions aux bords	34
4.5.3	Solution approchée	35
4.6	Cas d'un domaine avec une marche	37
4.7	Intérêt des éléments finis $P^1 - P^2$ par rapport aux éléments finis $P^1 - P^1$	40
5	Conclusion	43

Table des figures

1	Quelques résultats obtenus lors de ce compte-rendu à titre d'illustration.	1
2	Exemple de maillage avec les nœuds (sommets et milieux des triangles) associés à des éléments finis P^2	5
3	Illustrations des différents maillages pour diverses valeurs de h	7
4	Solution exacte $h = 0.2$	16
5	Solution approchée $h = 0.2$	16
6	Solution exacte $h = 0.1$	16
7	Solution approchée $h = 0.1$	16
8	Solution exacte $h = 0.05$	16
9	Solution approchée $h = 0.05$	16
10	Solution exacte $h = 0.025$	16
11	Solution approchée $h = 0.025$	16
12	Comparaison des erreurs L^2 et H^1 en fonction du maillage.	17
13	La solution approchée pour $h = 0.1$	22
14	Évolution des erreurs L^2 et H^1 en fonction du maillage (h)	23
15	Solution approchée $h = 0.025$	24
16	Évolution des erreurs L^2 et H^1 en fonction de la taille du maillage (h)	24
17	Maillage pour l'équation de Stokes.	27
18	Comparaison de la vitesse approchée et exacte	35
19	Représentation des différentes erreurs des grandeurs du problème de Stokes.	36
20	Représentation des différentes erreurs des grandeurs du problème de Stokes.	37
21	Représentation du maillage pour le problème de la marche.	38
23	Graphiques des variations de pression, u_1 , et u_2 pour différentes viscosités.	40
24	Vitesse horizontale U_1 dans le cas $P^1 - P^1$	41
25	Vitesse verticale U_1 dans le cas $P^1 - P^1$	42
26	Pression dans le cas $P^1 - P^1$	42

1 Introduction

Dans ce travail pratique, nous nous intéressons à l'écoulement d'un fluide visqueux incompressible et stationnaire dans un canal bidimensionnel, en régime caractérisé par un faible nombre de Reynolds. Ce problème est modélisé par les *équations de Stokes* :

$$\begin{aligned} -\nu \Delta \mathbf{u} + \nabla p &= \mathbf{f}, & \text{dans } \Omega, \\ \nabla \cdot \mathbf{u} &= 0, & \text{dans } \Omega, \end{aligned} \tag{1}$$

qui représentent une simplification des équations de Navier-Stokes. Cette simplification repose sur la négligence du terme de convection, considéré comme négligeable face au terme de diffusion dans ce régime.

L'objectif de ce TP est de progresser dans la résolution de ces équations, en surmontant les difficultés spécifiques qu'elles posent. En particulier, leur discrétisation nécessite l'utilisation d'éléments finis P^2 . Avant d'appliquer ces éléments finis aux équations de Stokes, nous les mettrons en œuvre sur le problème de Poisson étudié lors du TP 1.

Remarque : Ce rapport de TP a été fait dans l'optique d'être pouvoir lu et compris sans son énoncé, d'où des redites avec l'énoncé.

2 Éléments finis P^2 pour le problème de Poisson avec condition de Neumann

Soit Ω un ouvert borné de $\mathbb{R}^2$ à frontière polygonale et $f \in L^2(\Omega)$. On rappelle le problème du TP 1 : trouver $u \in H^1(\Omega)$ tel que

$$\begin{cases} u - \Delta u = f & \text{dans } \Omega, \\ \frac{\partial u}{\partial n} = 0 & \text{sur } \partial\Omega. \end{cases} \quad (2)$$

2.1 Discrétisation

Soit $\mathcal{T}_h$ une triangulation du domaine Ω et $(T_\ell)_{\ell=1,\dots,L}$ les triangles de $\mathcal{T}_h$. Nous considérons V_h , l'approximation de $H^1(\Omega)$ par des éléments finis P^2 associés à la triangulation $\mathcal{T}_h$:

$$V_h = \{v_h \in C^0(\Omega), v_h|_{T_\ell} \in P^2 \ \forall \ell = 1, \dots, L\},$$

où P^2 est l'espace des polynômes de degré au plus 2. Les fonctions de base sont associées aux sommets des triangles et aux milieux des arêtes (voir Figure 2). On note donc $(M_I)_{I=1,\dots,N}$ les nœuds du maillage (qui incluent les sommets et les milieux des arêtes), et $(w_I)_{I=1,\dots,N}$ la base de V_h définie par $w_I(M_J) = \delta_{IJ}$, $1 \leq I, J \leq N$.

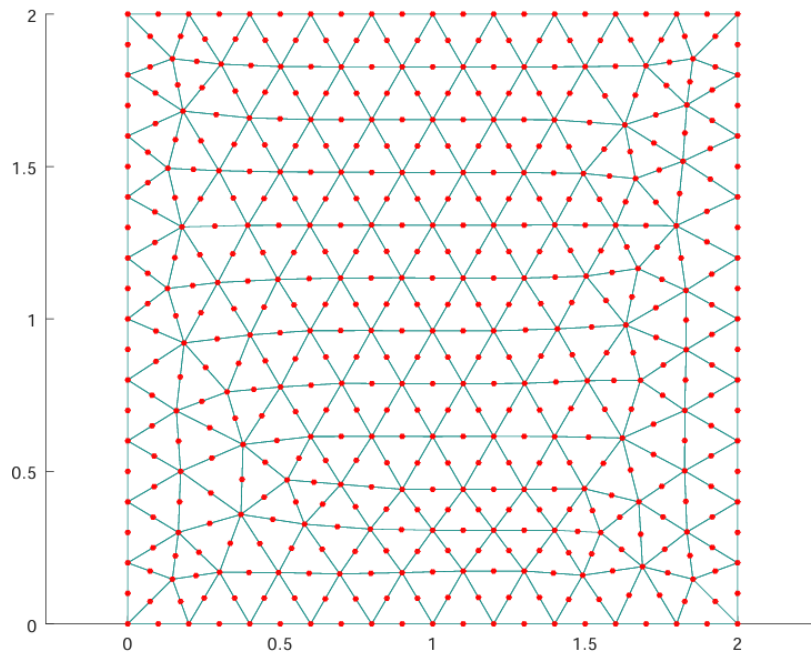


FIGURE 2 – Exemple de maillage avec les nœuds (sommets et milieux des triangles) associés à des éléments finis P^2 .

On rappelle que la formulation variationnelle démontrée lors du premier TP est :

Trouver $u \in H^1(\Omega)$ telle que :

$$\int_{\Omega} uv \, d\Omega + \int_{\Omega} \nabla u \cdot \nabla v \, d\Omega = \int_{\Omega} f v \, d\Omega \quad \forall v \in H^1(\Omega) \quad (\text{FV})$$

Ainsi, que sous sa forme de système linéaire :

$$(\mathbb{M} + \mathbb{K})\vec{\mathbf{U}} = \vec{\mathbf{L}}$$

avec $\mathbb{M}_{IJ} = \int_{\Omega} w_J w_I d\Omega$; la matrice de masse
 $\mathbb{K}_{IJ} = \int_{\Omega} \nabla w_J \cdot \nabla V_h d\Omega$; la matrice de rigidité

$$\vec{\mathbf{U}} = (u_h(S_I))_{I \in 1, \dots, N} \quad (3)$$

$$\vec{\mathbf{L}} = \left(\int_{\Omega} f w_I d\Omega \right)_{I \in 1, \dots, N} \quad (4)$$

2.2 Géométrie et maillage

Dans cette section, nous considérons le problème défini sur le domaine $\Omega = [0, 3] \times [0, 2]$. Afin de résoudre ce problème, nous avons créé des maillages d'ordre 2. Et pour étudier la convergence pour quatre pas de maillage différents : $h = 0.1$, $h = 0.2$, $h = 0.05$, et $h = 0.025$. Voici les étapes suivies pour générer et visualiser ces maillages :

2.2.1 Création des maillages

Pour générer les maillages d'ordre 2, nous avons suivi la procédure décrite dans le TP 1. Après avoir sélectionné l'option "2D", nous avons cliqué sur "Set order 2" avant de sauvegarder chaque maillage correspondant à un pas de maillage donné.

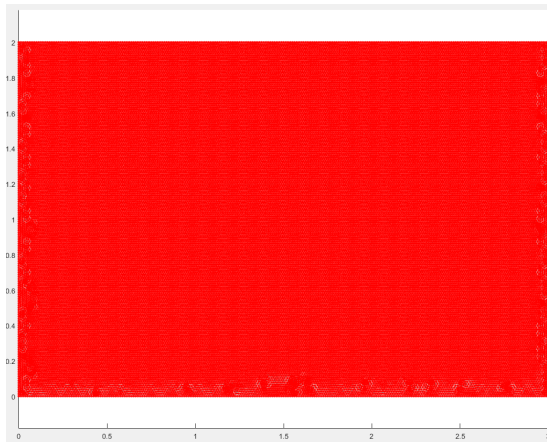
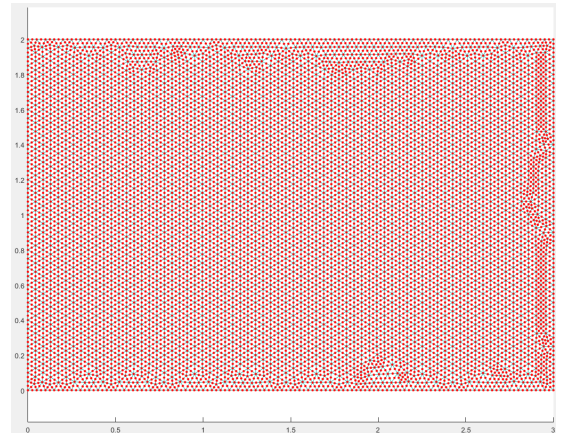
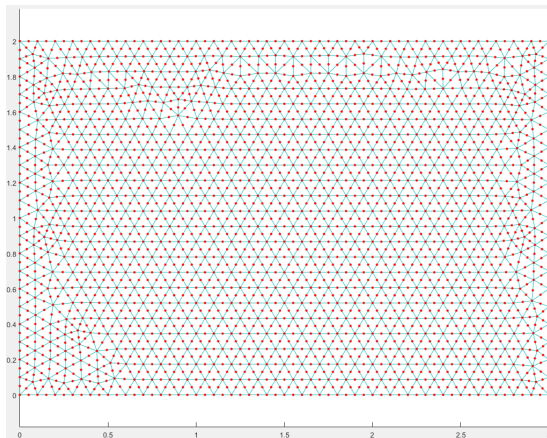
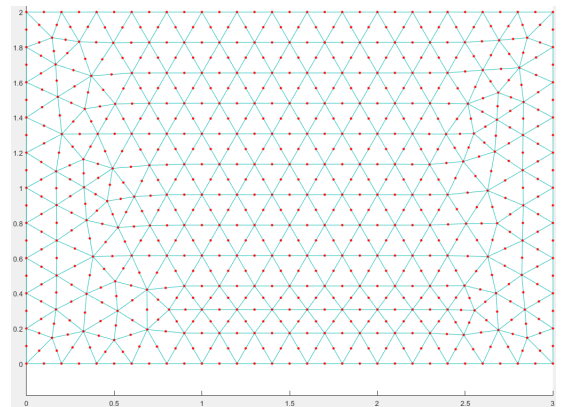
2.2.2 Structures de données du maillage

Les structures de données associées aux maillages sont construites à l'aide de la routine `lecture_msh_ordre2`. Cette routine permet d'extraire les informations suivantes :

- **Nbpt** : le nombre total de nœuds, incluant les sommets et les milieux des arêtes du maillage.
- **Coorneu** : un tableau de dimensions $\text{Nbpt} \times 2$, où chaque ligne contient les coordonnées (x_I, y_I) du nœud I .
- **Numtri** : un tableau de dimensions $L \times 6$, où L est le nombre de triangles. Chaque ligne de ce tableau contient les indices des 3 sommets suivis des indices des 3 milieux associés à un triangle donné.

2.2.3 Affichage des maillages

Les maillages créés pour chaque pas h ont été visualisés à l'aide de la routine `affichemaillage_ordre2.m`. Cette routine permet de représenter graphiquement les maillages afin de vérifier, par exemple, leur structure et leur densité en fonction de h .

(a) Maillage $h = 0.025$ (b) Maillage $h = 0.05$ (c) Maillage $h = 0.1$ (d) Maillage $h = 0.2$ FIGURE 3 – Illustrations des différents maillages pour diverses valeurs de h .

2.3 Calcul des matrices élémentaires par passage à l'élément fini de référence

Pour calculer les matrices élémentaires associées à un triangle T_ℓ , nous utilisons la méthode du passage à l'élément fini de référence. Ce procédé ramène le calcul des intégrales sur un triangle quelconque T_ℓ au triangle de référence $\hat{T}$, dont les sommets sont $\hat{M}_1 = (0,0)$, $\hat{M}_2 = (1,0)$, et $\hat{M}_3 = (0,1)$.

Les étapes du calcul sont décrites ci-dessous.

2.3.1 Définition des fonctions de base locales

Soit T_ℓ un triangle de sommets $M_1 = (x_1, y_1)$, $M_2 = (x_2, y_2)$, $M_3 = (x_3, y_3)$, et de milieux $M_4 = (x_4, y_4)$, $M_5 = (x_5, y_5)$, $M_6 = (x_6, y_6)$. Les fonctions de base locales associées aux coordonnées barycentriques du triangle sont données par :

$$\begin{aligned} w_1 &= \lambda_1(2\lambda_1 - 1), & w_4 &= 4\lambda_1\lambda_2, \\ w_2 &= \lambda_2(2\lambda_2 - 1), & w_5 &= 4\lambda_2\lambda_3, \\ w_3 &= \lambda_3(2\lambda_3 - 1), & w_6 &= 4\lambda_3\lambda_1. \end{aligned}$$

2.3.2 Transformation entre $\hat{T}$ et T_ℓ

La transformation affine $\mathcal{F}_\ell : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ permet de passer du triangle de référence $\hat{T}$ au triangle T_ℓ . Cette transformation est définie par :

$$\mathcal{F}_\ell(\hat{M}) = B_\ell \hat{M} + S_\ell,$$

où $B_\ell \in \mathcal{M}_2(\mathbb{R})$ est une matrice 2×2 et $S_\ell \in \mathbb{R}^2$ est un vecteur de translation.

Expression de B_ℓ et S_ℓ

Pour déterminer B_ℓ et S_ℓ , on impose que la transformation satisfait :

$$\mathcal{F}_\ell(\hat{M}_1) = M_1, \quad \mathcal{F}_\ell(\hat{M}_2) = M_2, \quad \mathcal{F}_\ell(\hat{M}_3) = M_3,$$

où :

$$\hat{M}_1 = (0, 0), \quad \hat{M}_2 = (1, 0), \quad \hat{M}_3 = (0, 1),$$

et $M_1 = (x_1, y_1)$, $M_2 = (x_2, y_2)$, $M_3 = (x_3, y_3)$ sont les sommets du triangle T_ℓ .

À partir de ces relations, on obtient :

$$S_\ell = M_1 = \begin{pmatrix} x_1 \\ y_1 \end{pmatrix},$$

et la matrice B_ℓ s'exprime comme :

$$B_\ell = \begin{pmatrix} x_2 - x_1 & x_3 - x_1 \\ y_2 - y_1 & y_3 - y_1 \end{pmatrix}.$$

Expression final de la transformation

Avec ces expressions de B_ℓ et S_ℓ , la transformation devient :

$$\mathcal{F}_\ell(\hat{M}) = \begin{pmatrix} x_2 - x_1 & x_3 - x_1 \\ y_2 - y_1 & y_3 - y_1 \end{pmatrix} \begin{pmatrix} \hat{x} \\ \hat{y} \end{pmatrix} + \begin{pmatrix} x_1 \\ y_1 \end{pmatrix}.$$

2.3.3 Formulation des intégrales

Les intégrales associées aux matrices de masse et de rigidité pour le triangle T_ℓ sont exprimées comme suit :

Intégrales de masse :

$$\int_{T_\ell} w_I(M) w_J(M) d\Omega = \int_{\hat{T}} \hat{w}_I(\hat{M}) \hat{w}_J(\hat{M}) |\det B_\ell| d\hat{\Omega}, \quad I, J = 1, \dots, 6.$$

Intégrales de rigidité :

$$\int_{T_\ell} \nabla w_I(M) \cdot \nabla w_J(M) d\Omega = \int_{\hat{T}} \left[(B_\ell^{-1})^\top \hat{\nabla} \hat{w}_I(\hat{M}) \right] \cdot \left[(B_\ell^{-1})^\top \hat{\nabla} \hat{w}_J(\hat{M}) \right] |\det B_\ell| d\hat{\Omega}.$$

Pour calculer les fonctions de base locales, nous avons besoin des coordonnées barycentriques λ_1 , λ_2 et λ_3 . En un point (x, y) du triangle, celles-ci sont données par :

$$\begin{aligned} \lambda_1(x, y) &= \frac{1}{D} (y_{23}(x - x_3) - x_{23}(y - y_3)), \\ \lambda_2(x, y) &= \frac{1}{D} (y_{31}(x - x_1) - x_{31}(y - y_1)), \\ \lambda_3(x, y) &= \frac{1}{D} (y_{12}(x - x_2) - x_{12}(y - y_2)), \end{aligned}$$

où $x_{ij} = x_i - x_j$, $y_{ij} = y_i - y_j$ pour i et j différents dans $\{1, 2, 3\}$, et $D = x_{23}y_{31} - x_{31}y_{23}$. Notons que D est égal, au signe près, à deux fois l'aire du triangle.

Rappel des expressions des coordonnées barycentriques λ_1 , λ_2 , et λ_3 du triangle $\hat{T}$

Commençons par calculer les différences nécessaires pour déterminer D :

$$x_{23} = x_2 - x_3 = 1 - 0 = 1, \quad y_{23} = y_2 - y_3 = 0 - 1 = -1,$$

$$x_{31} = x_3 - x_1 = 0 - 0 = 0, \quad y_{31} = y_3 - y_1 = 1 - 0 = 1.$$

L'aire du triangle est donc :

$$D = x_{23}y_{31} - x_{31}y_{23} = (1)(1) - (0)(-1) = 1.$$

On a le système suivant pour les coordonnées barycentriques $\hat{\lambda}_2$:

$$\begin{cases} \hat{\lambda}_2(x, 0) = x, \\ \hat{\lambda}_2(0, y) = 0, \\ \hat{\lambda}_2(0, 0) = 0. \end{cases}$$

Ainsi, $\boxed{\hat{\lambda}_2(x, y) = x}$.

Pour $\hat{\lambda}_3$, on a :

$$\begin{cases} \hat{\lambda}_3(x, 0) = 0, \\ \hat{\lambda}_3(0, y) = y, \\ \hat{\lambda}_3(0, 0) = 0. \end{cases}$$

Ainsi, on a que $\boxed{\hat{\lambda}_3(x, y) = y}$.

Enfin, $\hat{\lambda}_1(x, y) = 1 - x - y$.

On considère un triangle T avec les sommets :

$$M_1 = (x_1, y_1) = (0, 0), \quad M_2 = (x_2, y_2) = (1, 0), \quad M_3 = (x_3, y_3) = (0, 1).$$

Ainsi, la formule pour $\lambda_1(x, y)$ devient :

$$\hat{\lambda}_1(x, y) = \frac{1}{1} ((-1)(x - 0) - (1)(y - 1))$$

$$\hat{\lambda}_1(x, y) = -x - y + 1.$$

Finalement, nous avons la fonction barycentrique : $\boxed{\hat{\lambda}_1(x, y) = 1 - x - y}$.

Les intégrales du second membre dans les équations des intégrales de masse et de rigidité peuvent être calculées de manière analytique, mais pour les éléments finis de type P^2 , ces calculs deviennent laborieux. Pour les évaluer efficacement, nous allons plutôt utiliser *une formule de quadrature*.

De manière générale, une *formule de quadrature* à N_Q points est définie comme suit : étant donné une fonction G continue par morceaux de $\hat{T}$, l'intégrale de G sur $\hat{T}$ est approximée par :

$$\int_{\hat{T}} G d\hat{\Omega} \approx \sum_{q=1}^{N_Q} \hat{c}_q G(\hat{S}_q),$$

où $\hat{S}_q$ sont les points de quadrature dans $\hat{T}$, et $\hat{c}_q$ sont les poids positifs associés à ces points.

On dit qu'une formule de quadrature est d'ordre k si elle permet d'approximer de manière exacte les intégrales des polynômes de degré k . Pour l'évaluation des intégrales de masse et de rigidité, nous allons utiliser respectivement des formules de quadrature d'ordre 4 et d'ordre 2.

Ainsi, pour la matrice de masse, nous utiliserons la formule de quadrature à 6 points de Gauss-Legendre, qui est d'ordre 4, et qui est définie sur le triangle de référence $\hat{T}$ par :

$$\begin{aligned}\hat{S}_1 &= (0.0915762135098, 0.0915762135098), & \hat{c}^1 &= 0.05497587183, \\ \hat{S}_2 &= (0.81684757298, 0.0915762135098), & \hat{c}^2 &= 0.05497587183, \\ \hat{S}_3 &= (0.0915762135098, 0.81684757298), & \hat{c}^3 &= 0.05497587183, \\ \hat{S}_4 &= (0.1081030181681, 0.4459484909160), & \hat{c}^4 &= 0.116907948, \\ \hat{S}_5 &= (0.4459484909160, 0.1081030181681), & \hat{c}^5 &= 0.116907948, \\ \hat{S}_6 &= (0.4459484909160, 0.4459484909160), & \hat{c}^6 &= 0.116907948.\end{aligned}$$

Pour la matrice de rigidité, nous utiliserons la formule de quadrature à 3 points de Gauss-Legendre, qui est d'ordre 2, et qui est définie sur le triangle de référence T par :

$$\begin{aligned}S_1 &= \left(\frac{1}{6}, \frac{1}{6}\right), & c_1 &= \frac{1}{6}, \\ S_2 &= \left(\frac{2}{3}, \frac{1}{6}\right), & c_2 &= \frac{1}{6}, \\ S_3 &= \left(\frac{1}{6}, \frac{2}{3}\right), & c_3 &= \frac{1}{6}.\end{aligned}$$

2.3.4 Routine de la matrice de masse élémentaire

On remarque que pour les éléments finis quadratiques P^2 , chaque triangle possède 6 nœuds (3 aux sommets et 3 aux milieux des arêtes). Ainsi, les matrices élémentaires de masse $\mathbb{M}^l$ et de rigidité $\mathbb{K}^l$ sont de dimension 6×6 .

```

1 function [Mel] = matM_elem_p2(S1, S2, S3)
2 % préliminaires, pour faciliter la lecture:
3 x1 = S1(1); y1 = S1(2);
4 x2 = S2(1); y2 = S2(2);
5 x3 = S3(1); y3 = S3(2);
6
7 % calcul de la matrice de masse
8 % -----
9 % Initialisation
10
11 Mel = zeros(6, 6);
12 B = [x2 - x1, x3 - x1;
13      y2 - y1, y3 - y1];
14 if (abs(det(B)) <= eps)
15     error('l aire d un triangle est nulle!!!');
16 end;
17 detB = abs(det(B));
18
19 % Points et poids de quadrature
20 S_hat = [0.0915762135098, 0.0915762135098;
21          0.8168475729805, 0.0915762135098;
22          0.0915762135098, 0.8168475729805;
23          0.1081030181681, 0.4459484909160;
24          0.4459484909160, 0.1081030181681;
25          0.4459484909160, 0.4459484909160];
26 poids = [0.05497587183, 0.05497587183, 0.05497587183, 0.1116907948, 0.1116907948,
27          0.1116907948];
28 % Boucle sur les points de quadrature

```

```

29
30 for q = 1:length(poids)
31     % Coordonnées des points de quadrature
32     lambda1 = 1 - S_hat(q, 1) - S_hat(q, 2);      % Coordonnée barycentrique 1
33     lambda2 = S_hat(q, 1);                        % Coordonnée barycentrique 2
34     lambda3 = S_hat(q, 2);                        % Coordonnée barycentrique 3
35     % On exprime w sous forme d'un vecteur afin d'all ger le calcul
36     w = [lambda1 * (2 * lambda1 - 1);
37          lambda2 * (2 * lambda2 - 1);
38          lambda3 * (2 * lambda3 - 1);
39          4 * lambda1 * lambda2;
40          4 * lambda2 * lambda3;
41          4 * lambda1 * lambda3];
42     Mel = Mel + poids(q) * (w * w') * detB ;
43 end

```

Listing 1 – Calcul de la matrice de masse élémentaire en P_2 Lagrange

Le code que nous avons écrit calcule la matrice de masse élémentaire pour un élément fini de type P^2 Lagrange dans le cadre de la méthode des éléments finis. Nous commençons par prendre en entrée les coordonnées des trois sommets du triangle S_1 , S_2 et S_3 . La matrice de masse Mel est initialisée comme une matrice nulle de taille 6×6 , puis nous construisons cette matrice en utilisant une quadrature de Gauss sur les points du triangle. Les coordonnées barycentriques et les poids associés à chaque point de quadrature sont utilisés pour calculer la contribution de chaque point à la matrice de masse. Nous vérifions également que le déterminant de la matrice B est non nul, afin de garantir que l'aire du triangle est valide. Enfin, nous mettons à jour la matrice de masse élémentaire Mel en fonction des contributions de chaque point de quadrature et la retournons comme résultat.

2.3.5 Routine de la matrice de rigidité élémentaire

Avant de commencer le code de matrice de rigidité élémentaire, nous allons calculer le gradient des fonctions de base.

Pour un triangle de référence, les coordonnées barycentriques sont définies comme suit :

$$\lambda_1 = 1 - x - y, \quad \lambda_2 = x, \quad \lambda_3 = y. \quad (5)$$

Les fonctions de forme quadratiques sont exprimées en fonction de ces coordonnées barycentriques :

$$\begin{aligned}
 w_1 &= \lambda_1(2\lambda_1 - 1), \\
 w_2 &= \lambda_2(2\lambda_2 - 1), \\
 w_3 &= \lambda_3(2\lambda_3 - 1), \\
 w_4 &= 4\lambda_1\lambda_2, \\
 w_5 &= 4\lambda_2\lambda_3, \\
 w_6 &= 4\lambda_1\lambda_3.
 \end{aligned}$$

Calcul des dérivées partielles En utilisant la définition des coordonnées barycentriques et des fonctions de forme, nous calculons les dérivées partielles suivantes.

Dérivées de w_1

$$w_1 = \lambda_1(2\lambda_1 - 1) = (1 - x - y)(2(1 - x - y) - 1).$$

En dérivant par rapport à x et y , nous obtenons :

$$\frac{\partial w_1}{\partial x} = -3 + 4x + 4y, \quad \frac{\partial w_1}{\partial y} = -3 + 4x + 4y.$$

Dérivées de w_2

$$w_2 = \lambda_2(2\lambda_2 - 1) = x(2x - 1).$$

$$\frac{\partial w_2}{\partial x} = 4x - 1, \quad \frac{\partial w_2}{\partial y} = 0.$$

Dérivées de w_3

$$w_3 = \lambda_3(2\lambda_3 - 1) = y(2y - 1).$$

$$\frac{\partial w_3}{\partial x} = 0, \quad \frac{\partial w_3}{\partial y} = 4y - 1.$$

Dérivées de w_4

$$w_4 = 4\lambda_1\lambda_2 = 4(1 - x - y)x.$$

$$\frac{\partial w_4}{\partial x} = 4(1 - 2x - y), \quad \frac{\partial w_4}{\partial y} = -4x.$$

Dérivées de w_5

$$w_5 = 4\lambda_2\lambda_3 = 4xy.$$

$$\frac{\partial w_5}{\partial x} = 4y, \quad \frac{\partial w_5}{\partial y} = 4x.$$

Dérivées de w_6

$$w_6 = 4\lambda_1\lambda_3 = 4(1 - x - y)y.$$

$$\frac{\partial w_6}{\partial x} = -4y, \quad \frac{\partial w_6}{\partial y} = 4(1 - x - 2y).$$

Gradient des fonctions de forme Nous regroupons les résultats sous forme matricielle pour obtenir le gradient complet des fonctions de forme (puis ensuite on mets dans la routine de la matrice de rigidité élémentaire :

$$\nabla w_i = \begin{bmatrix} \frac{\partial w_i}{\partial x} & \frac{\partial w_i}{\partial y} & \frac{\partial w_i}{\partial x} & \frac{\partial w_i}{\partial y} & \frac{\partial w_i}{\partial x} & \frac{\partial w_i}{\partial y} \end{bmatrix}.$$

Soit ci-dessous, le code allégé de la matricé de rigidité élémentaire :

```

1 function [Kel] = matK_elem_p2(S1, S2, S3)
2
3 % preliminaires, pour faciliter la lecture:
4 x1 = S1(1); y1 = S1(2);
5 x2 = S2(1); y2 = S2(2);
6 x3 = S3(1); y3 = S3(2);
7
8 % calcul de la matrice de rigidité
9 % -----
10 % Initialisation
11 Kel = zeros(6, 6); % A COMPLETER
12
13 % Points et poids de quadrature
14 S_hat = [1/6, 1/6;
15          2/3, 1/6;
16          1/6, 2/3];
17 poids = 1/6;
18
19 %Traitement de B1 pour calculer la transpos e de son inverse utilis e dans
20 % le calcul de quadrature, tout comme son d terminant
21 B = [x2 - x1, x3 - x1;
22      y2 - y1, y3 - y1];
23 invB = inv(B);
24 detB = abs(det(B));
25
26 for q = 1:size(S_hat, 1)
27     xsi = S_hat(q, 1);
28     eta = S_hat(q, 2);

```

```

29 %Apr s avoir calcul chaque gradient de chaque fonction de bases wi, on
30 %fait des 6 gradients trouv s une matrice 6x6 afin de calcul la matrice
31 % lmentaire via un produit de matrices
32 gradN_hat = [-3 + 4*xsi + 4*eta, 4*xsi - 1, 0, 4*(1 - 2*xsi - eta), 4*eta,
33             -4*eta;
34             -3 + 4*xsi + 4*eta, 0, 4*eta - 1, -4*xsi, 4*xsi, 4*(1 - xsi - 2*eta)];
35 gradN = invB' * gradN_hat;
36 % Contribution la matrice de rigidit
37 Kel = Kel + poids * (gradN' * gradN) * detB ;
end

```

Listing 2 – Calcul de la matrice de rigidité élémentaire en P_2 Lagrange

Le code que nous avons développé calcule la matrice de rigidité élémentaire pour un élément fini de type P^2 Lagrange dans le cadre de la méthode des éléments finis. Nous commençons par prendre en entrée les coordonnées des trois sommets du triangle S_1 , S_2 , et S_3 , puis nous initialisons la matrice de rigidité Kel à zéro. Nous définissons ensuite les points de quadrature et les poids associés pour effectuer l'intégration sur l'élément. Pour chaque point de quadrature, nous calculons le gradient des fonctions de base dans le système de coordonnées locales du triangle, puis nous appliquons une transformation inverse de la matrice B pour obtenir les gradients dans le système global. Nous utilisons ces gradients pour contribuer à la matrice de rigidité élémentaire Kel , en multipliant les gradients transposés et les gradients eux-mêmes, et en prenant en compte le déterminant de la matrice B . Enfin, la matrice de rigidité élémentaire Kel est retournée après avoir "intégré" sur tous les points de quadrature.

2.4 Assemblage

Maintenant, nous allons assembler la matrice $\mathbb{A} = \mathbb{M} + \mathbb{K}$. Pour cela, dans la routine `principal_neumann_p2.m` on a :

```

1  for l=1:Nbtri
2
3      % Coordonnees des sommets du triangles
4      %De m me que dans le TP1
5      S1 = Coorneu(Numtri(l, 1), :);
6      S2 = Coorneu(Numtri(l, 2), :);
7      S3 = Coorneu(Numtri(l, 3), :);
8
9      % Calcul des matrices elementaires du triangle l
10     Kel=matK_elem_p2(S1, S2, S3);
11     Mel=matM_elem_p2(S1, S2, S3);
12
13     % On fait l'assemblage des matrices globales
14     % De m me que dans le TP1
15     for i=1:6
16         I= Numtri(l,i);
17         for j=1:6
18             J=Numtri(l,j);
19             MM(I,J)= MM(I,J)+Mel(i,j);
20             KK(I,J)= KK(I,J)+Kel(i,j);
21         end %boucle j
22     end %boucle i
23
24 end % boucle l
25
26 AA=MM+KK;

```

Listing 3 – Calcul de la matrice de masse élémentaire en P_2 Lagrange

Dans ce code, nous procédons à l'assemblage des matrices de masse et de rigidité élémentaires pour chaque élément du maillage. Nous commençons par itérer sur chaque élément du maillage,

identifié par l . Pour chaque élément, nous extrayons les coordonnées des trois sommets du triangle en utilisant les indices de la matrice *Numtri* et les stockons dans les variables $S1$, $S2$, et $S3$. Ensuite, nous calculons les matrices de masse et de rigidité élémentaires associées à cet élément en appelant respectivement les fonctions `matM_elem_p2` et `matK_elem_p2`. Ces matrices élémentaires sont ensuite assemblées dans les matrices globales MM et KK à l'aide de deux boucles imbriquées qui parcourent les indices des fonctions de forme locales. L'assemblage est effectué en accumulant les valeurs de chaque élément dans la position correspondante des matrices globales. Après avoir traité tous les éléments du maillage, nous additionnons la matrice de masse MM et la matrice de rigidité KK pour obtenir la matrice globale AA , qui sera utilisée dans la résolution du système d'équations pour le problème donné.

Si la fonction source f est continue, on doit l'interpoler dans l'espace des éléments finis. L'interpolation $\pi_h f$ est exprimée comme une combinaison linéaire des fonctions de base w_i :

$$\pi_h f = \sum_{j=1}^N f(S_j) w_j$$

où S_j sont les nœuds du maillage et $f(S_j)$ les valeurs de f aux nœuds.

Chaque composante L_i du vecteur second membre peut être approximée par :

$$L_i \approx \int_{\Omega} \pi_h f \cdot w_i d\Omega = \sum_{j=1}^N f(S_j) \int_{\Omega} w_j w_i d\Omega$$

L'intégrale $\int_{\Omega} w_j w_i d\Omega$ correspond aux éléments de la matrice de masse M . Ainsi, on peut écrire :

$$\mathbf{L} \approx M\mathbf{F}$$

où $\mathbf{F} = [f(S_1), f(S_2), \dots, f(S_N)]^T$ est le vecteur des valeurs de f aux nœuds.

Dans le cas général, le vecteur $\mathbf{L}$ est calculé en multipliant la matrice de masse M par le vecteur $\mathbf{F}$, contenant les valeurs interpolées de la fonction source f :

$$\mathbf{L} \approx M\mathbf{F}$$

2.5 Validation

Pour valider notre code, nous allons tester sur une solution connue.

Soit $u(x, y) = 3 \cos(\pi x) \cos(2\pi y)$. Nous voulons vérifier que u satisfait la condition au bord, c'est-à-dire que la dérivée de u par rapport à la normale au bord est nulle.

Le domaine est $[0, 3] \times [0, 2]$, avec quatre bords : - $x = 0$, $x = 3$ (bords verticaux), - $y = 0$, $y = 2$ (bords horizontaux).

Bords où $x = 0$ ou $x = 3$ (normale dans la direction x) :

La dérivée de u par rapport à x est :

$$\frac{\partial u}{\partial x} = -3\pi \sin(\pi x) \cos(2\pi y)$$

- Pour $x = 0$: $\frac{\partial u}{\partial x}(0, y) = 0$, - Pour $x = 3$: $\frac{\partial u}{\partial x}(3, y) = 0$.

Bords où $y = 0$ ou $y = 2$ (normale dans la direction y) :

La dérivée de u par rapport à y est :

$$\frac{\partial u}{\partial y} = -6\pi \cos(\pi x) \sin(2\pi y)$$

- Pour $y = 0$: $\frac{\partial u}{\partial y}(x, 0) = 0$, - Pour $y = 2$: $\frac{\partial u}{\partial y}(x, 2) = 0$.

Conclusion

La dérivée de u par rapport à la normale au bord est nulle sur tous les bords du domaine, ce qui montre que $u(x, y) = 3 \cos(\pi x) \cos(2\pi y)$ satisfait la condition au bord.

Soit $u(x, y) = 3 \cos(\pi x) \cos(2\pi y)$, la fonction donnée. Le Laplacien de u est donné par :

$$\Delta u = -15\pi^2 \cos(\pi x) \cos(2\pi y)$$

Ainsi :

$$\Delta u - u = -15\pi^2 \cos(\pi x) \cos(2\pi y) - 3 \cos(\pi x) \cos(2\pi y)$$

Ce qui simplifie à :

$$u - \Delta u = (15\pi^2 + 3) \cos(\pi x) \cos(2\pi y) = f(x, y)$$

Dans la routine `f.m`, on a que :

```
1 function val = f(x,y)
2 val = (15 * pi^2 + 3) .* cos(pi * x) .* cos(2 * pi * y);
3 end
```

Listing 4 – Routine `f.m`

Ainsi, on peut calculer les différentes solutions et les comparer avec les solutions exactes. À première vue, les solutions approchées sont correctes, mais pour avoir un jugement plus rigoureux et précis, nous allons calculer les erreurs relatives en norme L^2 et en semi-norme H^1 .

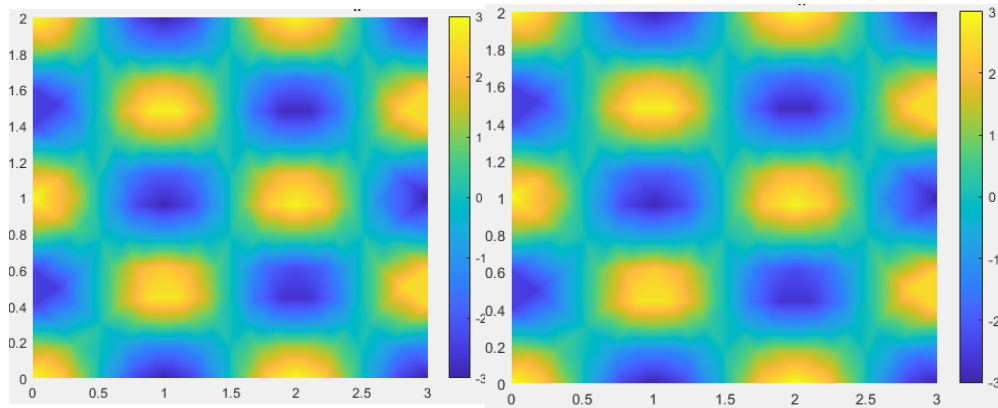


FIGURE 4 – Solution exacte $h = 0.2$

FIGURE 5 – Solution approchée $h = 0.2$

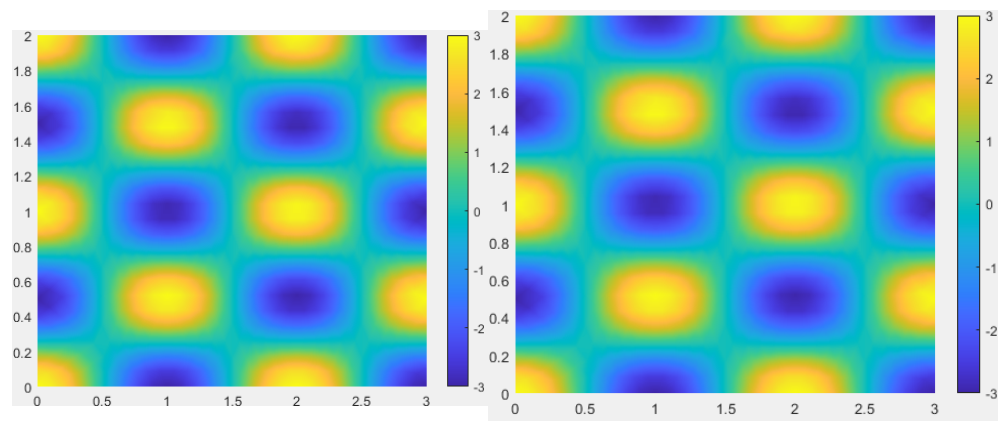


FIGURE 6 – Solution exacte $h = 0.1$

FIGURE 7 – Solution approchée $h = 0.1$

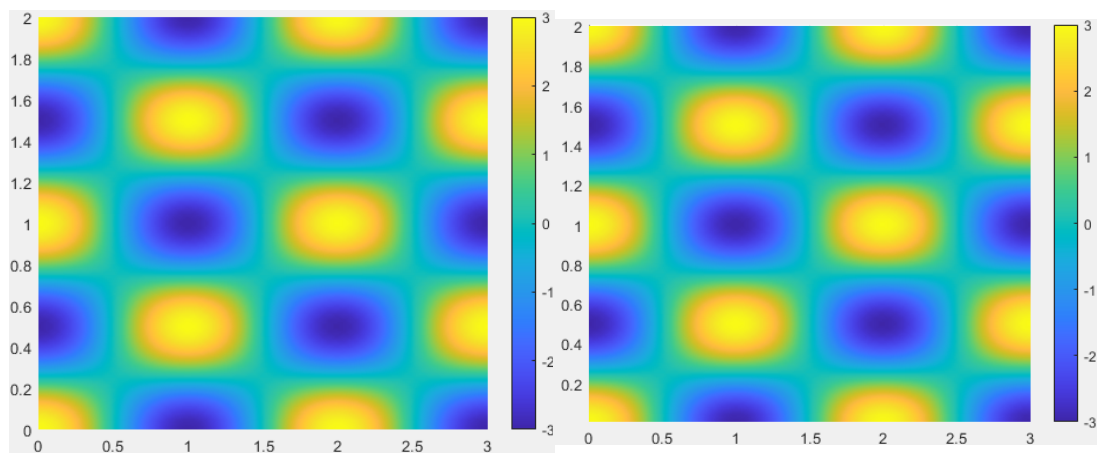


FIGURE 8 – Solution exacte $h = 0.05$

FIGURE 9 – Solution approchée $h = 0.05$

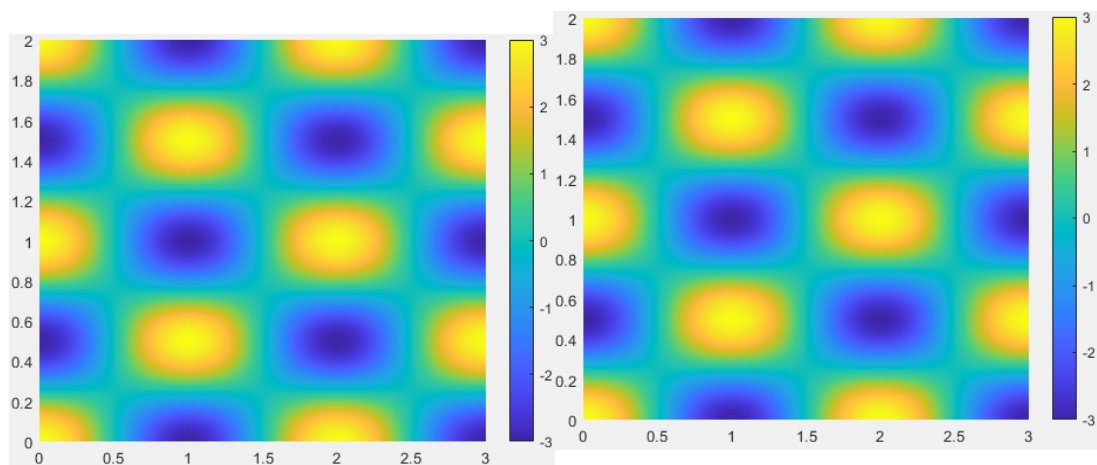


FIGURE 10 – Solution exacte $h = 0.025$

FIGURE 11 – Solution approchée $h = 0.025$

Pour nous intéresser aux différents types d'erreurs. Pour cela, nous reprenons une partie de notre code du premier TP et nous obtenons une courbe pour l'erreur L^2 et une autre pour l'erreur H^1 . En effet, nous avons la même structure de code car les erreurs sont calculées de la même façon :

Pour la norme L^2 :

$$\|u - u_h\|_{L^2(\Omega)} \approx \|\pi_h u - u_h\|_{L^2(\Omega)} = \sqrt{\int_{\Omega} |\pi_h u - u_h|^2 d\Omega}$$

Puis :

$$\|u - u_h\|_{L^2(\Omega)} \approx \sqrt{\sum_{i=1}^{N_{\text{som}}} \sum_{j=1}^{N_{\text{som}}} (u_{\text{exact}}(S_i) - u_h(S_i)) \mathbf{M}_{ij} (u_{\text{exact}}(S_j) - u_h(S_j))} = \sqrt{(U_{\text{exact}} - U)^T \mathbf{M} (U_{\text{exact}} - U)}$$

Comme $\frac{\|u - u_h\|_{L^2(\Omega)}}{\|u\|_{L^2(\Omega)}}$ est l'erreur normalisée, on doit donc calculer $\|u\|_{L^2(\Omega)}$. On utilise la même méthode que précédemment et on obtiens :

$$\|u\|_{L^2(\Omega)} \approx \|\pi_h u\|_{L^2(\Omega)} = \sqrt{U_{\text{exact}}^T \mathbf{M} U_{\text{exact}}}$$

Pour la norme H^1 :

L'erreur peut aussi être évaluée à travers la semi-norme H^1 , en calculant l'erreur sur le gradient :

$$\|u - u_h\|_{H^1} = \|\nabla u - \nabla u_h\|_{L^2(\Omega)}^2 \approx \|\nabla \pi_h u - \nabla u_h\|_{L^2(\Omega)}^2 = \sqrt{(U_{\text{exact}} - U)^T \mathbf{K} (U_{\text{exact}} - U)}$$

Enfin, de façon analogue, la norme H^1 de u est donnée par :

$$\|u\|_{H^1} \approx \|\pi_h u\|_{H^1} = \sqrt{U_{\text{exact}}^T \mathbf{K} U_{\text{exact}}}$$

Ainsi, nous pouvons tracer la figure 14.

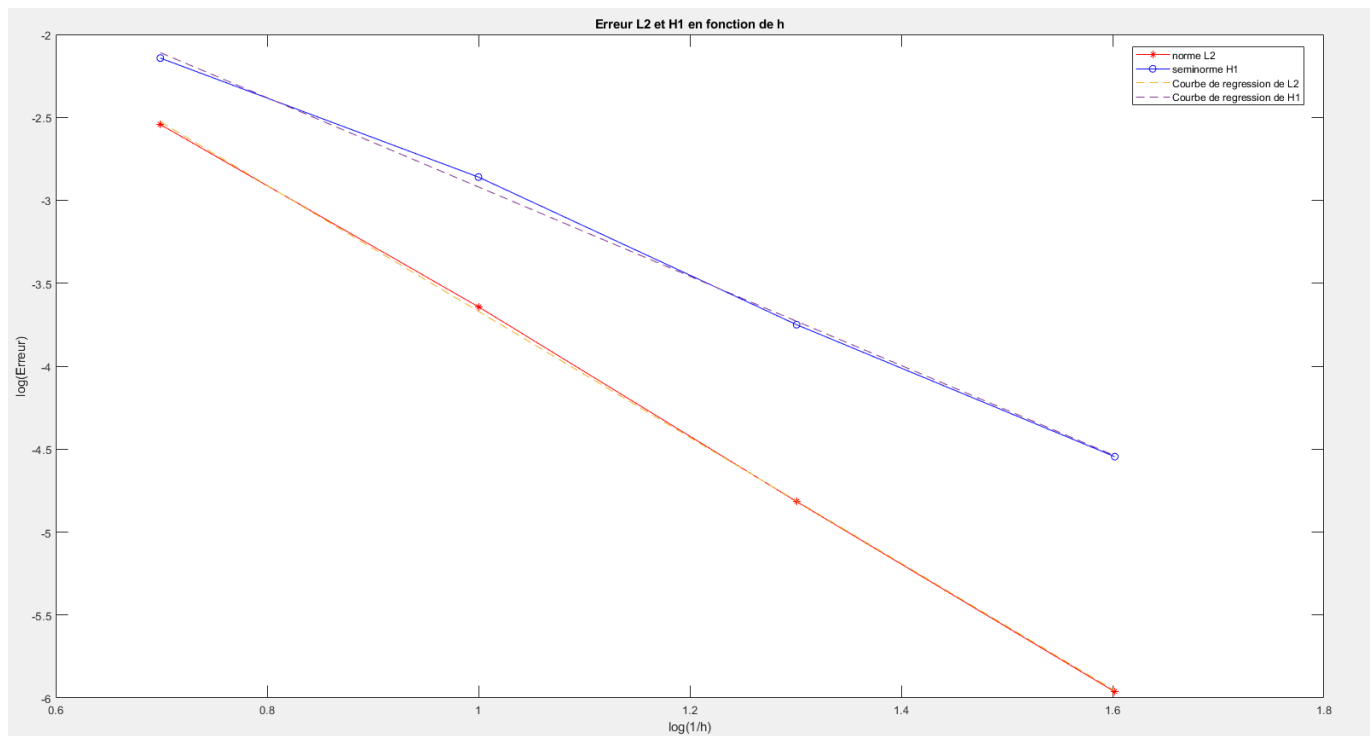


FIGURE 12 – Comparaison des erreurs L^2 et H^1 en fonction du maillage.

TABLE 1 – Résumé des erreurs L^2 et H^1 en fonction du pas h .

Pas h	Erreur L^2	Erreur relative L^2	Erreur H^1	Erreur relative H^1
0.2	0.007426	0.002864	0.131363	0.007203
0.1	0.000593	0.000228	0.025199	0.001381
0.05	0.000040	0.000015	0.003246	0.000178
0.025	0.000003	0.000001	0.000520	0.000028

Pente de la régression pour l'erreur L^2 : -3.798155

Pente de la régression pour l'erreur H^1 : -2.690556

À travers notre analyse des résultats présentés, nous constatons plusieurs éléments concernant la méthode numérique que nous avons mise en œuvre pour résoudre les équations de Stokes. En observant les erreurs $\|\cdot\|_{L^2}$ et $\|\cdot\|_{H^1}$ en fonction de la taille du maillage h , nous remarquons que ces erreurs diminuent lorsque h diminue, c'est-à-dire lorsque le maillage est affiné. Cela traduit directement une amélioration de la précision de la solution approchée, comme nous nous y attendions.

Nous savons que ce comportement est conforme aux résultats théoriques des éléments finis, qui prédisent des ordres de convergence spécifiques pour ces normes. En particulier, nous avons constaté que la norme L^2 , qui quantifie l'erreur globale sur le domaine, décroît plus rapidement que la semi-norme H^1 , qui évalue l'erreur sur les dérivées de la solution. Cette différence dans les taux de décroissance reflète le choix des fonctions de base que nous avons utilisé dans notre méthode.

En affinant le maillage, nous avons pu observer une amélioration progressive de la précision des résultats. Cependant, nous avons également pris conscience que cet accroissement de précision entraîne une augmentation importante des coûts computationnels. Cela nous amène à souligner un compromis incontournable : nous devons choisir avec soin le niveau de raffinement du maillage pour garantir une précision adéquate tout en maintenant des temps de calcul raisonnables.

Au premier TP, nous avons trouvé un ordre de convergence d'environ 2.052 selon la norme L^2 et 1.713 pour la norme H^1 . Dans ce TP, nous avons trouvé des ordres de convergence supérieur (3.79 et 2.69 respectivement), ce qui était attendu car dans le TP1, nous avions des éléments finis de P^1 alors qu'ici les éléments finis sont de P^2 .

En conclusion, grâce à notre étude, nous avons pu vérifier que le schéma numérique utilisé respecte les propriétés théoriques attendues. Nous avons ainsi confirmé expérimentalement la fiabilité et l'efficacité de la méthode. Ce travail nous a permis de mettre en lumière l'importance du choix des paramètres de discrétisation dans la quête d'un équilibre entre précision et efficacité computationnelle.

3 Éléments finis P2 pour le problème de Poisson avec condition de Dirichlet

On cherche maintenant à résoudre le problème suivant :

$$\begin{cases} u - \Delta u = f & \text{dans } \Omega, \\ u = 0 & \text{sur } \partial\Omega. \end{cases}$$

La formulation variationnelle de ce problème s'écrit : Trouver $u \in H_0^1(\Omega)$ tel que

$$\int_{\Omega} uv \, d\Omega + \int_{\Omega} \nabla u \cdot \nabla v \, d\Omega = \int_{\Omega} f v \, d\Omega, \quad \forall v \in H_0^1(\Omega).$$

3.1 Discrétisation

Comme $V_h^0 \subset H_0^1(\Omega)$, on peut réécrire la formulation variationnelle dans V_h^0 :
Trouver $u_h \in V_h^0$ telle que :

$$\int_{\Omega} (u_h v_h + \nabla u_h \cdot \nabla v_h) \, d\Omega = \int_{\Omega} f v_h \, d\Omega, \quad \forall v_h \in V_h^0.$$

La solution approchée $u_h(x, y)$ s'écrit sous la forme :

$$u_h(x, y) = \sum_{j=1}^{N_0} u_h(M_j) w_j(x, y), \quad \forall (x, y) \in \bar{\Omega}.$$

En substituant cette expression dans la formulation variationnelle discrète et en prenant $v_h = w_I$ pour $I = 1, \dots, N_0$, on obtient :

$$\sum_{j=1}^{N_0} U_j^0 \int_{\Omega} w_j w_I \, d\Omega + \sum_{j=1}^{N_0} U_j^0 \int_{\Omega} \nabla w_j \cdot \nabla w_I \, d\Omega = \int_{\Omega} f w_I \, d\Omega, \quad \forall I = 1, \dots, N_0.$$

On reconnaît les matrices et le vecteur suivants :

$$M_{IJ}^0 = \int_{\Omega} w_I w_J \, d\Omega, \quad K_{IJ}^0 = \int_{\Omega} \nabla w_I \cdot \nabla w_J \, d\Omega, \quad L_I^0 = \int_{\Omega} f w_I \, d\Omega.$$

Ainsi, le système linéaire s'écrit sous la forme :

$$A^0 U^0 = L^0, \quad \text{où } A^0 = M^0 + K^0.$$

Propriétés des matrices

— **Matrice de masse** M^0 : Elle est définie positive, symétrique, et creuse :

$$\mathbf{a}^T M^0 \mathbf{a} = \int_{\Omega} \left(\sum_{I=1}^{N_0} a_I w_I \right)^2 \, d\Omega > 0, \quad \forall \mathbf{a} \neq 0.$$

— **Matrice de rigidité** K^0 : Elle est semi-définie positive, symétrique, et creuse :

$$\mathbf{a}^T K^0 \mathbf{a} = \int_{\Omega} \left| \sum_{I=1}^{N_0} a_I \nabla w_I \right|^2 \, d\Omega \geq 0.$$

— **Matrice A^0** : En combinant les propriétés de M^0 et K^0 , A^0 est définie positive et symétrique. Par conséquent, le système admet une solution unique.

On considère que les nœuds situés sur la frontière $\partial\Omega$ sont numérotés de $N_0 + 1$ à N tandis que les nœuds à l'intérieur sont numérotés de 1 à N_0 . L'espace discret est alors défini par :

$$V_h^0 = \text{Vect}(w_1, \dots, w_{N_0}).$$

Le système simplifié, ayant la même dimension que le système initial, s'écrit :

$$\tilde{A}\tilde{U} = \tilde{L},$$

avec :

$$\tilde{A} = \begin{bmatrix} \tilde{A}_{B,B} & \tilde{A}_{B,I} \\ \tilde{A}_{I,B} & \tilde{A}_{I,I} \end{bmatrix}, \quad \tilde{L} = \begin{bmatrix} \tilde{L}_B \\ \tilde{L}_I \end{bmatrix}.$$

Sous les conditions initiales, on impose $\tilde{L}_I = 0$. En outre, on adopte les conventions suivantes :

$$\tilde{A}_{B,I} = \tilde{A}_{I,B} = 0, \quad \tilde{A}_{I,I} = \text{Id}, \quad \tilde{A}_{B,B} = A^0.$$

Par conséquent, les matrices réduites s'écrivent comme suit :

$$\tilde{A} = \begin{bmatrix} A^0 & 0 \\ 0 & \text{Id} \end{bmatrix}, \quad \tilde{L} = \begin{bmatrix} L^0 \\ 0 \end{bmatrix}.$$

Dans cette question, nous allons présenter le code nécessaire pour la construction des matrices ainsi que l'implémentation de la méthode de pseudo-élimination dans le fichier `elimine.m`.

```

1 % Assemblage des matrices
2 MM = sparse(Nbpt, Nbpt); % Matrice de masse
3 KK = sparse(Nbpt, Nbpt); % Matrice de rigidité
4 LL = zeros(Nbpt, 1); % Vecteur second membre
5
6 for l = 1:Nbtri
7     S1 = Coorneu(Numtri(l, 1), :);
8     S2 = Coorneu(Numtri(l, 2), :);
9     S3 = Coorneu(Numtri(l, 3), :);
10
11     Kel = matK_elem_p2(S1, S2, S3);
12     Mel = matM_elem_p2(S1, S2, S3);
13
14     for i = 1:6
15         I = Numtri(l, i);
16         for j = 1:6
17             J = Numtri(l, j);
18             MM(I, J) = MM(I, J) + Mel(i, j);
19             KK(I, J) = KK(I, J) + Kel(i, j);
20         end
21     end
22 end
23
24 AA = MM + KK; % Matrice du syst me
25 FF = f(Coorneu(:, 1), Coorneu(:, 2));
26 LL = MM * FF; % Second membre sans condition de bord

```

Listing 5 – La construction des matrices

```

1 function [tilde_AA, tilde_LL] = elimine(AA, LL, Refneu, Coorneu)
2     Idx = find(Refneu==1);
3     tilde_AA = AA;
4     tilde_LL = LL;
5

```

```

6      %%Conditon de Dirichlet non homogene
7      tilde_AA(Idc,:) = 0;
8      tilde_LL(Idc) = g(Coorneu(Idc,1),Coorneu(Idc,2));
9
10
11      [N,~] = size(Idc);
12      for i=1:N
13          id = Idc(i);
14          tilde_AA(id,id) = 1;
15      end
16  end

```

Listing 6 – Routine elimine.m

3.2 Validation

3.2.1 Vérifier les conditions aux bords

Tout comme avec les conditions de Neumann, nous allons valider notre code en partie par le biais d'une solution connue. Vérifions si la solution $u(x, y) = 3 \sin(\pi x) \sin(\pi y)$ satisfait bien les conditions aux bords imposées.

1. Lorsque $x = 0$ ou $x = 3$, on a :

$$u(x, y) = 3 \sin(\pi x) \sin(\pi y) = 3 \cdot 0 \cdot \sin(\pi y) = 0.$$

2. Lorsque $y = 0$ ou $y = 2$, on a :

$$u(x, y) = 3 \sin(\pi x) \sin(\pi y) = 3 \cdot \sin(\pi x) \cdot 0 = 0.$$

Dans les deux cas, $u(x, y)$ est nul sur le bord $\partial\Omega$. Par conséquent, la fonction $u(x, y)$ satisfait bien la condition aux limites $u = 0$ sur $\partial\Omega$.

3.2.2 Calcule f

Pour $u(x, y) = 3 \sin(\pi x) \sin(\pi y)$, on calcule le terme source f .

$$\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}.$$

En prenant les dérivées secondes :

$$\frac{\partial^2 u}{\partial x^2} = -3\pi^2 \sin(\pi x) \sin(\pi y), \quad \frac{\partial^2 u}{\partial y^2} = -3\pi^2 \sin(\pi x) \sin(\pi y).$$

Ainsi,

$$\Delta u = -6\pi^2 \sin(\pi x) \sin(\pi y).$$

En utilisant l'équation $f = u - \Delta u$, on obtient :

$$f = 3 \sin(\pi x) \sin(\pi y) - (-6\pi^2 \sin(\pi x) \sin(\pi y)).$$

Simplification :

$$f = (3 + 6\pi^2) \sin(\pi x) \sin(\pi y).$$

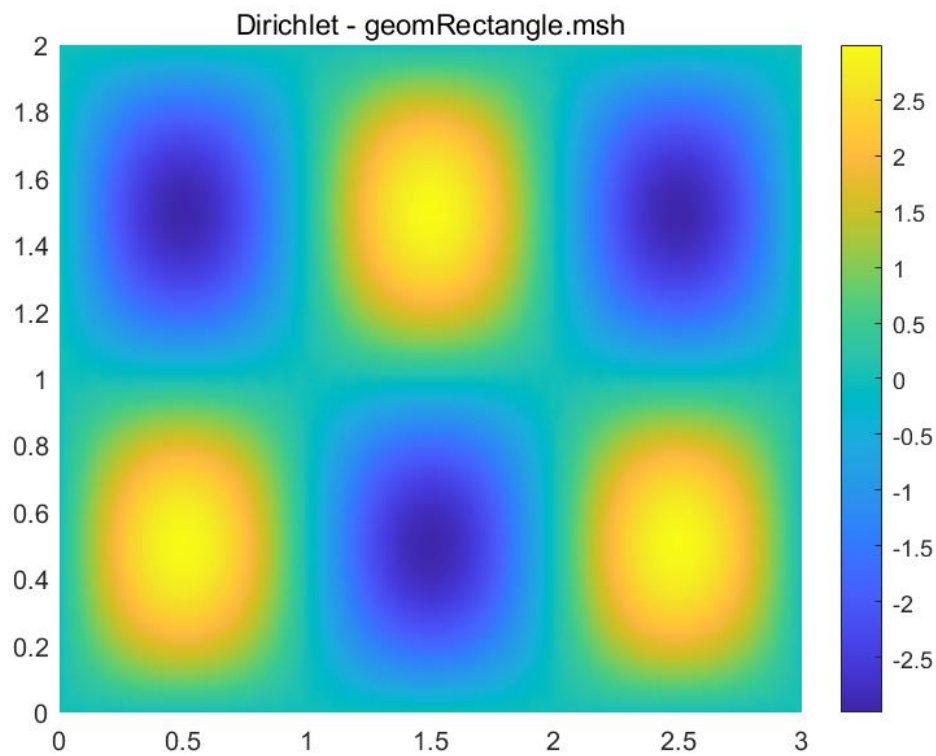
Le fichier `f.m` ainsi que le graphe de la solution approchée sont donnés ci-dessous.

```

1  function val = f(x,y)
2
3  val = (3 + 6 * pi^2) .* sin(pi * x) .* sin(pi * y);
4
5  end

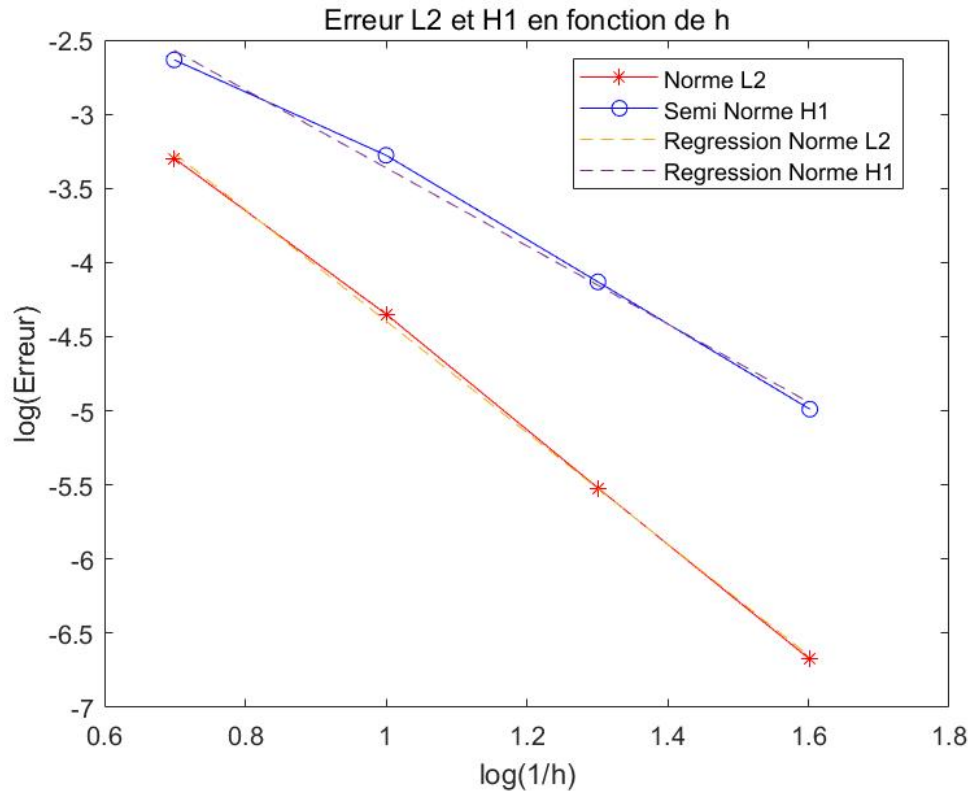
```

Listing 7 – Routine f.m

FIGURE 13 – La solution approchée pour $h = 0.1$.

3.2.3 Analyse des erreurs

Grâce à la routine précédente et avec les mêmes calculs que précédemment, on peut estimer la valeur de l'erreur L^2 et H^1 et ainsi tracer les graphes de convergences ci-dessous. De plus, une régression linéaire permet d'obtenir l'ordre de convergence de la méthode (on l'inscrit dans la légende du graphe).

FIGURE 14 – Évolution des erreurs L^2 et H^1 en fonction du maillage (h)

Après avoir effectué une régression linéaire sur les points de données obtenus, nous avons obtenu les valeurs de pente correspondantes comme suit :

- **Pente de la régression pour l'erreur L^2 :** -3.756353
- **Pente de la régression pour l'erreur H^1 :** -2.631515

1. **Erreur L^2 (norme L^2) :**

- Le graphe montre une décroissance relativement lente de l'erreur avec h .
- Cela indique que la convergence en norme L^2 est actuellement d'ordre 3.76, ce qui est conforme aux observations, bien que la valeur théorique soit de 2.

2. **Erreur H^1 (semi-norme H^1) :**

- La semi-norme H^1 diminue également avec h , mais de manière moins marquée que la norme L^2 .
- Cela indique que la convergence en norme H^1 est actuellement d'ordre 2.63, ce qui est conforme aux observations, bien que la valeur théorique soit de 1.

3.3 Conditions de Dirichlet non homogènes

Pour valider le code, on utilise une solution analytique connue donnée par :

$$u(x, y) = 3 \cos(\pi x) \cos(2\pi y),$$

avec $(x, y) \in \bar{\Omega} = [0, 3] \times [0, 2]$. Nous allons déterminer le terme source $f(x, y)$ ainsi que la condition de Dirichlet $g(x, y)$, puis vérifier si le code produit les résultats attendus.

1. Calcul de $f(x, y)$

L'équation de Poisson est donnée par :

$$u - \Delta u = f.$$

Pour $u(x, y) = 3 \cos(\pi x) \cos(2\pi y)$, on a :

$$\frac{\partial^2 u}{\partial x^2} = -3\pi^2 \cos(\pi x) \cos(2\pi y), \quad \frac{\partial^2 u}{\partial y^2} = -12\pi^2 \cos(\pi x) \cos(2\pi y).$$

Ainsi :

$$\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = -3\pi^2 \cos(\pi x) \cos(2\pi y) - 12\pi^2 \cos(\pi x) \cos(2\pi y),$$

$$\Delta u = -15\pi^2 \cos(\pi x) \cos(2\pi y).$$

En utilisant $f = u - \Delta u$, on trouve :

$$f(x, y) = 3 \cos(\pi x) \cos(2\pi y) - (-15\pi^2 \cos(\pi x) \cos(2\pi y)).$$

$$f(x, y) = (3 + 15\pi^2) \cos(\pi x) \cos(2\pi y).$$

2. Condition de Dirichlet $g(x, y)$

Sur le bord de $\partial\Omega$, la condition de Dirichlet non homogène est donnée par :

$$g(x, y) = u(x, y) = 3 \cos(\pi x) \cos(2\pi y), \quad (x, y) \in \partial\Omega.$$

3. Implémentation en code

```
1 function val = f(x, y)
2     % Terme source pour u(x, y) = 3 * cos(pi * x) * cos(2 * pi * y)
3     val = (3 + 15 * pi^2) * cos(pi * x) .* cos(2 * pi * y);
4 end
```

Listing 8 – Routine f.m

```
1 function g_values = g(Coorneu, Idx)
2     % Extraire les coordonnées des nœuds spécifiques
3     x = Coorneu(Idx, 1); % Coordonnées x
4     y = Coorneu(Idx, 2); % Coordonnées y
5
6     % Calculer les valeurs de g
7     g_values = g_expression(x, y);
8 end
```

Listing 9 – Routine g.m

3.3.1 Validation

On obtient ainsi la solution suivante pour $h = 0,025$ et on trace le graphe de convergence de la même manière que précédemment.

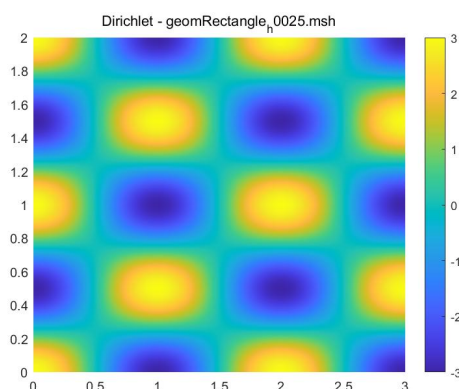


FIGURE 15 – Solution approchée $h = 0.025$

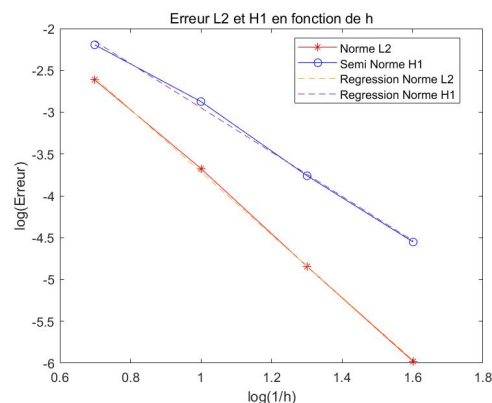


FIGURE 16 – Évolution des erreurs L^2 et H^1 en fonction de la taille du maillage (h)

Conclusion : Dans cet partie, nous avons étudié un problème de Poisson avec une condition de Dirichlet. Le dernier problème consiste à trouver une fonction $u \in H^1(\Omega)$ satisfaisant $u - \Delta u = f$ dans Ω et $u = g$ sur $\partial\Omega$. Nous avons formulé la version variationnelle de ce problème et utilisé une méthode d'élimination des conditions de Dirichlet pour adapter le système linéaire en fonction des valeurs imposées g .

Nous avons obtenu une solution numérique, et analysé la convergence en termes des erreurs dans les normes L^2 et H^1 . Les résultats montrent une décroissance linéaire des logarithmes des erreurs relatives en fonction de la taille du maillage, confirmant la cohérence de la méthode. Nous avons également validé la solution numérique en la comparant à une solution exacte $u(x, y) = 3 \cos(\pi x) \cos(2\pi y)$, et observé une excellente concordance.

En conclusion, cet exercice met en évidence l'efficacité de la méthode des éléments finis pour résoudre des équations aux dérivées partielles avec des conditions aux limites non homogènes. Les résultats obtenus confirment la convergence et la précision de la méthode, tout en montrant qu'elle est robuste et facilement adaptable à des situations plus complexes.

4 Équation de Stokes

Nous introduisons ici le **problème de Stokes**. Considérons un domaine ouvert borné $\Omega \subset \mathbb{R}^2$ à frontière polygonale $\Gamma = \partial\Omega$. La frontière Γ est décomposée en deux parties disjointes, $\Gamma = \Gamma_D \cup \Gamma_N$, avec $\Gamma_N \neq \emptyset$. Une condition de Dirichlet est spécifiée sur Γ_D , tandis qu'une condition de Neumann est imposée sur Γ_N .

L'objectif est de déterminer la **vitesse** $\mathbf{u} = \mathbf{u}(x, y) = (u_1(x, y), u_2(x, y))$ et la **pression** $p = p(x, y)$, qui vérifient les équations de Stokes suivantes : trouver $\mathbf{u} \in [H^1(\Omega)]^2$ et $p \in H^1(\Omega)$ tels que

$$\begin{cases} -\nu \Delta \mathbf{u} + \nabla p = 0 & \text{dans } \Omega, \\ -\mathbf{u} = 0 & \text{dans } \Omega, \\ \mathbf{u} = \mathbf{g} & \text{sur } \Gamma_D, \\ \nu \frac{\partial \mathbf{u}}{\partial \mathbf{n}} - p \mathbf{n} = 0 & \text{sur } \Gamma_N, \end{cases} \quad (6)$$

où ν est la viscosité dynamique (supposée constante), et $\mathbf{g}$ représente la donnée de Dirichlet sur Γ_D . Par ailleurs, $\mathbf{n}$ désigne la normale unitaire extérieure à $\partial\Omega$.

Le système (9) peut être reformulé composante par composante de la manière suivante :

$$\begin{cases} -\nu \Delta u_1 + \frac{\partial p}{\partial x} = 0 & \text{dans } \Omega, \\ -\nu \Delta u_2 + \frac{\partial p}{\partial y} = 0 & \text{dans } \Omega, \\ -\frac{\partial u_1}{\partial x} - \frac{\partial u_2}{\partial y} = 0 & \text{dans } \Omega, \\ (u_1, u_2) = (g_1, g_2) & \text{sur } \Gamma_D, \\ \nu \nabla u_1 \cdot \mathbf{n} - p n_1 = 0 & \text{sur } \Gamma_N, \\ \nu \nabla u_2 \cdot \mathbf{n} - p n_2 = 0 & \text{sur } \Gamma_N. \end{cases} \quad (7)$$

Avec ces équations, nous pouvons déduire la formulation variationnelle du problème afin de l'écrire sous forme linéaire pour pouvoir ensuite résoudre le problème numériquement.

Première équation de Stokes : Partons de l'équation

$$-\nu \Delta \mathbf{u} + \nabla p = 0 \quad \text{dans } \Omega.$$

Multiplions cette équation vectorielle par une fonction test $\mathbf{v} \in [H_{0,D}^1(\Omega)]^2$ et intégrons sur Ω :

$$\int_{\Omega} (-\nu \Delta \mathbf{u} + \nabla p) \cdot \mathbf{v} \, d\Omega = 0.$$

Comme $p \in H^1(\Omega)$, donc $\nabla p \in L^2(\Omega)$ donc $\Delta u \in L^2(\Omega)$. Dans ce cours, nous avons supposé que cela implique que $\mathbf{u} \in [H^2(\Omega)]^2$. Les hypothèses de la formule de Green sont donc vérifiées. En appliquant la formule de Green pour le terme $-\nu \Delta \mathbf{u}$, on obtient :

$$\int_{\Omega} -\nu \Delta \mathbf{u} \cdot \mathbf{v} \, d\Omega = \int_{\Omega} \nu \nabla \mathbf{u} : \nabla \mathbf{v} \, d\Omega - \int_{\partial\Omega} \nu \frac{\partial \mathbf{u}}{\partial \mathbf{n}} \cdot \mathbf{v} \, d\Gamma.$$

Pour le terme $\nabla p \cdot \mathbf{v}$, en appliquant également la formule de Green (car $p \in H^1(\Omega)$ donc $\nabla p \in L^2(\Omega)$, et que $\mathbf{v} \in [H_{0,D}^1(\Omega)]^2$), on a :

$$\int_{\Omega} \nabla p \cdot \mathbf{v} \, d\Omega = - \int_{\Omega} p \, \nabla \cdot \mathbf{v} \, d\Omega + \int_{\partial\Omega} p \mathbf{n} \cdot \mathbf{v} \, d\Gamma.$$

Ainsi, la première équation devient :

$$\int_{\Omega} \nu \nabla \mathbf{u} : \nabla \mathbf{v} \, d\Omega - \int_{\Omega} p \, \nabla \cdot \mathbf{v} \, d\Omega + \int_{\partial\Omega} \left(-\nu \frac{\partial \mathbf{u}}{\partial \mathbf{n}} + p \mathbf{n} \right) \cdot \mathbf{v} \, d\Gamma = 0.$$

Sur Γ_N , la condition de Neumann homogène impose que $-\nu \frac{\partial \mathbf{u}}{\partial \mathbf{n}} + p \mathbf{n} = 0$, ce qui annule le terme de bord sur Γ_N . Sur Γ_D , $\mathbf{v} = 0$. Il reste donc :

$$\int_{\Omega} \nu \nabla \mathbf{u} : \nabla \mathbf{v} \, d\Omega - \int_{\Omega} p \, \nabla \cdot \mathbf{v} \, d\Omega = 0, \quad \forall \mathbf{v} \in [H_{0,D}^1(\Omega)]^2.$$

Deuxième équation de Stokes, la condition d'incompressibilité : La deuxième équation de Stokes impose que $-\nabla \cdot \mathbf{u} = 0$ dans Ω . En multipliant par une fonction test $q \in L^2(\Omega)$ et en intégrant sur Ω , on obtient :

$$\int_{\Omega} (-\nabla \cdot \mathbf{u}) q \, d\Omega = 0 \quad \implies \quad \int_{\Omega} (-\nabla \cdot \mathbf{u}) q \, d\Omega = 0, \quad \forall q \in L^2(\Omega).$$

De plus, la donnée de Dirichlet impose que $\mathbf{u}|_{\Gamma_D} = \mathbf{g}$.

Formulation variationnelle complète : En combinant les deux équations obtenues ci-dessus, la formulation variationnelle s'écrit :

Trouver $\mathbf{u} = (u_1, u_2) \in [H^1(\Omega)]^2$ avec $\mathbf{u}|_{\Gamma_D} = \mathbf{g}$ et $p \in L^2(\Omega)$ tels que

$$\begin{cases} \nu \int_{\Omega} \nabla \mathbf{u} : \nabla \mathbf{v} \, d\Omega - \int_{\Omega} p \, \nabla \cdot \mathbf{v} \, d\Omega = 0, \quad \forall \mathbf{v} = (v_1, v_2) \in [H_{0,D}^1(\Omega)]^2 \\ - \int_{\Omega} \mathbf{u} \cdot \nabla q \, d\Omega = 0, \quad \forall q \in L^2(\Omega), \end{cases} \quad (8)$$

où l'on a noté $\nabla \mathbf{u} : \nabla \mathbf{v} = \nabla u_1 \cdot \nabla v_1 + \nabla u_2 \cdot \nabla v_2$ et

$$[H_{0,D}^1(\Omega)]^2 = \{\mathbf{v} \in [H^1(\Omega)]^2 \text{ tel que } \mathbf{v}|_{\Gamma_D} = (0, 0)\}.$$

Le théorème de Lax-Milgram ne peut pas être utilisé ici, mais on pourrait démontrer que la formulation variationnelle est bien posée. Étant donné que cela dépasse largement le cadre de ce TP, nous ne détaillerons pas cet aspect

4.1 Maillage

Dans le fichier `geomRectangle_partie3.geo`, la définition des éléments physiques (Physical) permetrons par la suite d'imposer des conditions au bord différentes sur les 4 côtés du rectangle .

Notamment, les lignes du fichier :

```
"Physical Line(1) = 4;
Physical Line(2) = 1, 3;
Physical Line(3) = 2;"
```

Où, on remarquera que `Physical Line(2)` fait référence aux lignes 1 et 3 du rectangle en même temps. Cela aura pour effet, de modifier le fichier `Refneu`, de tel sorte que si point d'un bord appartient à la "`Physical line(i)`" alors sa valeur sera `i` dans `Refneu`, et 0 s'il n'appartient pas au bord. On peut ensuite faire un fichier `mesh` à partir de `geo` pour obtenir un maillage comme montré sur la figure 17.

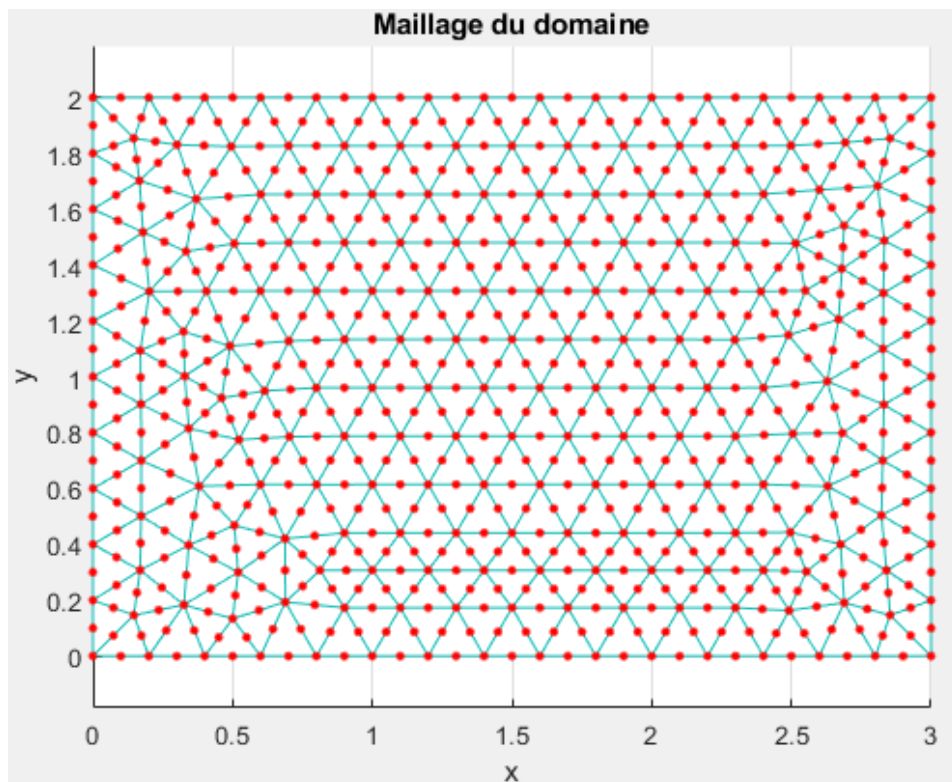


FIGURE 17 – Maillage pour l'équation de Stokes.

4.2 Discrétisation et calcul des matrices élémentaires

Pour discrétiser, nous allons le faire sur chaque composante de la vitesse grâce aux éléments finis P^2 . Tandis que pour la pression cela se fera grâce aux éléments finis P^1 . Nous aurons donc une paire d'éléments finis notée $P^2 - P^1$ nommée élément fini de Taylor-Hood. Nous pouvons supposer que les sommets des triangles sont numérotés de 1 à N_s , et les milieux des arêtes de N_s+1 à N . De plus, nous poserons que le vecteur $\vec{U}$ des inconnues du système linéaire issu de la discrétisation de (??) est un vecteur de $\mathbb{R}^{2N+N_s}$:

$$\vec{U} = [u_{1_h}(M_1), \dots, u_{1_h}(M_N), u_{2_h}(M_1), \dots, u_{2_h}(M_N), p_h(M_1), \dots, p_h(M_{N_s})]^T,$$

constitué des inconnues relatives à la première composante de la vitesse, puis à la seconde composante de la vitesse, puis à la pression. La matrice du système linéaire peut alors s'écrire par blocs

sous la forme

$$\mathbb{A} = \begin{bmatrix} \nu\mathbb{K} & \mathbf{0} & \mathbb{E} \\ \mathbf{0} & \nu\mathbb{K} & \mathbb{F} \\ \mathbb{G} & \mathbb{H} & \mathbf{0} \end{bmatrix} \in \mathbb{R}^{(2N+N_s) \times (2N+N_s)}$$

où

- Les deux blocs $\nu\mathbb{K} \in \mathbb{R}^{N \times N}$ contiennent les contributions issues de la discrétisation des termes $\nu \int_{\Omega} \nabla u_1 \cdot \nabla v_1 \, d\Omega$ et $\nu \int_{\Omega} \nabla u_2 \cdot \nabla v_2 \, d\Omega$. En d'autres termes, on a

$$\mathbb{K}_{IJ} = \int_{\Omega} \nabla w_I \cdot \nabla w_J \, d\Omega,$$

avec $(w_I)_{I=1,N}$ les fonctions de base P^2 utilisées dans les parties précédentes.

- Le bloc rectangulaire $\mathbb{E} \in \mathbb{R}^{N \times N_s}$ contient les contributions issues de la discrétisation du terme $-\int_{\Omega} p \frac{\partial v_1}{\partial x} \, d\Omega$, soit

$$\mathbb{E}_{IJ} = - \int_{\Omega} \tilde{w}_J \frac{\partial w_I}{\partial x} \, d\Omega,$$

avec $(\tilde{w}_J)_{J=1,N_s}$ les fonctions de base P^1 utilisées dans le TP 1.

- Le bloc rectangulaire $\mathbb{F} \in \mathbb{R}^{N \times N_s}$ contient les contributions issues de la discrétisation du terme $-\int_{\Omega} p \frac{\partial v_2}{\partial y} \, d\Omega$, à savoir

$$\mathbb{F}_{IJ} = - \int_{\Omega} \tilde{w}_J \frac{\partial w_I}{\partial y} \, d\Omega.$$

- Le bloc rectangulaire $\mathbb{G} \in \mathbb{R}^{N_s \times N}$ contient les contributions issues de la discrétisation du terme

$$- \int_{\Omega} \frac{\partial u_1}{\partial x} q \, d\Omega = 0, \quad \forall q \in L^2(\Omega),$$

à savoir (d'après notamment formule variationnelle et l'interpolation de u)

$$\mathbb{G}_{IJ} = - \int_{\Omega} \tilde{w}_I \frac{\partial w_J}{\partial x} \, d\Omega = E_{IJ}^{\top}$$

- Le bloc rectangulaire $\mathbb{H} \in \mathbb{R}^{N_s \times N}$ contient les contributions issues de la discrétisation du terme

$$- \int_{\Omega} \frac{\partial u_2}{\partial y} q \, d\Omega = 0, \quad \forall q \in L^2(\Omega)$$

à savoir (d'après notamment formule variationnelle et l'interpolation de u)

$$\mathbb{H}_{IJ} = - \int_{\Omega} \tilde{w}_I \frac{\partial w_J}{\partial y} \, d\Omega = F_{IJ}^{\top}$$

Ensuite, nous allons écrire deux routines `matE_elem.m` et `matM_elem.m` qui permettront d'obtenir respectivement les matrices élémentaires $\mathbb{E}^{\ell} \in \mathbb{R}^{6 \times 3}$ et $\mathbb{F}^{\ell} \in \mathbb{R}^{6 \times 3}$ à partir des coordonnées des 3 sommets d'un triangle $\mathcal{T}_\ell$.

Nous remarquons notamment que $\tilde{w}_J|_{\mathcal{T}_\ell} = \lambda_J$. De plus, nous retiendrons le fait que les formules suivantes, obtenues par passage à l'élément fini de référence, sont : pour $I = 1, \dots, 6$, $J = 1, \dots, 3$, le coefficient $(\mathbb{E}^{\ell})_{IJ}$ peut s'écrire

$$- \int_{\mathcal{T}_\ell} \lambda_J(M) \frac{\partial w_I(M)}{\partial x} \, d\Omega = - \int_{\hat{\mathcal{T}}} \hat{\lambda}_J(\hat{M}) \text{ligne}_1 \left((\mathcal{B}_\ell^T)^{-1} \hat{\nabla} \hat{w}_I(\hat{M}) \right) |\det \mathcal{B}_\ell| \, d\hat{\Omega},$$

et le coefficient $(\mathbb{F}^{\ell})_{IJ}$ peut s'écrire

$$- \int_{\mathcal{T}_\ell} \lambda_J(M) \frac{\partial w_I(M)}{\partial y} \, d\Omega = - \int_{\hat{\mathcal{T}}} \hat{\lambda}_J(\hat{M}) \text{ligne}_2 \left((\mathcal{B}_\ell^T)^{-1} \hat{\nabla} \hat{w}_I(\hat{M}) \right) |\det \mathcal{B}_\ell| \, d\hat{\Omega},$$

où l'on a utilisé la notation $\text{ligne}_m(\mathbb{B})$ pour indiquer la m -ième ligne d'une matrice $\mathbb{B}$. Pour calculer les intégrales sur $\hat{T}$ ci-dessus, on utilise les formules de quadrature à 3 points de Gauss-Legendre qui est d'ordre 2. Car, nous avons $\tilde{w}_J|_{\tau_l} = \lambda_J$, et que $\tilde{w}_J$ est un polynôme d'ordre 1 multiplié par le gradient d'un polynôme d'ordre 2 $\hat{w}_J$, c'est-à-dire un polynôme d'ordre 1. Donc, le produit donne un polynôme d'ordre 2. Enfin, en choisissant la formule de quadrature d'ordre 2, nous approchons de façon exacte les intégrales des polynômes d'ordre 2. La formule est définie sur le triangle de référence $\hat{T}$ par

$$\begin{aligned} S_1 &= \left(\frac{1}{6}, \frac{1}{6}\right), & c_1 &= \frac{1}{6}, \\ S_2 &= \left(\frac{2}{3}, \frac{1}{6}\right), & c_2 &= \frac{1}{6}, \\ S_3 &= \left(\frac{1}{6}, \frac{2}{3}\right), & c_3 &= \frac{1}{6}. \end{aligned}$$

tel que

$$\int_{\hat{T}} G d\hat{\Omega} \approx \sum_{q=1}^{N_Q} \hat{c}_q G(\hat{S}_q),$$

où $\hat{S}_q$ sont les points de quadrature dans $\hat{T}$, et $\hat{c}_q$ sont les poids positifs associés à ces points.

Pour calculer numériquement les matrices $\mathbb{E}^{\leq}$ et $\mathbb{F}^{\leq}$ élémentaires, nous reprenons le code pour le problème de Poisson avec condition de Neumann en modifiant ce qu'il faut.

```

1 function [Fel] = matE_elem(S1, S2, S3)
2 % preliminaires, pour faciliter la lecture:
3 x1 = S1(1); y1 = S1(2);
4 x2 = S2(1); y2 = S2(2);
5 x3 = S3(1); y3 = S3(2);
6
7 % Points et poids de quadrature
8 S_hat = [1/6, 1/6;
9          2/3, 1/6;
10         1/6, 2/3];
11 poids = 1/6;
12
13 % calcul de la matrice elementaire du bloc rectangulaire (p, dv1/dx)
14 % -----
15 Fel = zeros(6,3);
16 %Traitement de Bl pour calculer la transpos e de son inverse utilis e dans
17 % le calcul de quadrature, tout comme son d terminant
18 B = [x2 - x1, x3 - x1;
19      y2 - y1, y3 - y1];
20 if (abs(det(B)) <= eps)
21     error('1 aire d un triangle est nulle!!!');
22 end;
23 detB = abs(det(B));
24 invB = inv(B);
25 invBT=invB';
26 L2 = invBT(2,:);% Ici, nous prenons seulement la deuxi me ligne de l'inverse
    transpos e de Bl
27
28 for q = 1:size(S_hat, 1)
29     xsi = S_hat(q, 1);
30     eta = S_hat(q, 2);
31     % On reprends le TP1 pour les fonctions de base P1, on remarque que le
32     % D du TP1 est gal au detB. Dans le TP1, les fonctions de base taient
33     % gales aux coordonn es barycentriques.
34     %Donc, on peut crire que

```

```

35 %      lambda1 = 1 - S_hat(q, 1) - S_hat(q, 2); % Coordonn e barycentrique 1
36 %      lambda2 = S_hat(q, 1); % Coordonn e barycentrique 2
37 %      lambda3 = S_hat(q, 2); % Coordonn e barycentrique 3
38 % Pour simplifier all ger le code, nous mettons ceci sous forme d'un
39 % vecteur w_P1 (dans l'optique de faire un produit scalaire
40 w_P1= [1 - S_hat(q, 1) - S_hat(q, 2);S_hat(q, 1);S_hat(q, 2)];
41
42 %Apr s avoir calcul chaque gradient de chaque fonction de bases wi, on
43 %fait des 6 gradients trouv s une matrice 6x6 afin de calcul la matrice
44 % lmentaire via un produit de matrices
45 gradN_hat = [-3 + 4*xsi + 4*eta, 4*xsi - 1, 0, 4*(1
               - 2*xsi - eta), 4*eta, -4*eta;
46               -3 + 4*xsi + 4*eta, 0, 4*eta - 1, -4*xsi, 4*xsi, 4*(1 - xsi - 2*eta)];
47 wb = w_P1*L2 ;
48 % Contribution la matrice Fel
49 Fel = Fel + poids *(-(wb * gradN_hat))* detB ;
50 end

```

Pour la routine de $\mathbb{E}^<$, on fait de même, mais juste en modifiant

```

1 L2 = invBT(2,:);

```

par

```

1 L1 = invBT(1,:);

```

4.3 Assemblage

Passons maintenant à l'assemblage. Dans le fichier `principal_stokes.m`, l'assemblage de la matrice par blocs $\mathbb{A}$ est effectué avant la pseudo-élimination. En particulier, pour l'assemblage des blocs rectangulaires $P^2 - P^1$, il est important de noter que dans `Coorneu`, les sommets ne sont *pas* numérotés avant les milieux. L'assemblage est effectué en utilisant des matrices temporaires `EEtmp` et `FFtmp` de taille $N \times N$, et en extrayant ensuite les colonnes correspondantes aux sommets. La structure `numsommets`, construite dans le fichier `principal_stokes.m`, contient la liste des numéros des sommets, et peut donc être utilisée pour cette extraction (cf. les commentaires dans `principal_stokes.m`).

Le code MATLAB correspondant est le suivant :

```

1 for l=1:Nbtri
2
3     % Coordonnees des sommets du triangle, on utilise la m me technique que
4     % lors du premier exercice
5     % Rappel : Coorneu = matrice (Nbpt x 2) dont les composantes sont des r els
6     % qui
7     % repr sentent les coordonn es (x,y) des sommets du maillage.
8     S1=Coorneu(Numtri(l,1),:);
9     S2=Coorneu(Numtri(l,2),:);
10    S3=Coorneu(Numtri(l,3),:);
11
12    % On met 1 dans les entrees de isavertex correspondants aux 3 sommets
13    isavertex(Numtri(l,1)) = 1;
14    isavertex(Numtri(l,2)) = 1;
15    isavertex(Numtri(l,3)) = 1;
16
17    % Calcul des matrices elementaires du triangle l
18    Mel = matM_elem_p2(S1, S2, S3);
19    Kel = matK_elem_p2(S1, S2, S3);
20    Eel = matE_elem(S1, S2, S3);
21    Fel = matF_elem(S1, S2, S3);
22
23    % On fait l'assemblage des blocs (u_i, v_i), en suivant le sch ma de

```



```

23 % l'exercice 1
24 for i=1:6
25     I=Numtri(1,i);
26     for j=1:6
27         J=Numtri(1,j);
28         KK(I,J)=KK(I,J)+Kel(i,j);
29     end
30 end
31 % On fait l'assemblage des blocs (p, v1) et (p, v2), en suivant le schéma de
32 % l'exercice 1 avec une plage d'indice différente
33 for i=1:6
34     I=Numtri(1,i);
35     for j=1:3
36         J=Numtri(1,j);
37         EEtmp(I,J)=EEtmp(I,J)+Eel(i,j);
38         FFtmp(I,J)=FFtmp(I,J)+Fel(i,j);
39     end
40 end
41 end % for l
42
43 % find(isavertex~=0) donne la liste des numeros des sommets
44 numssommets = find(isavertex==1);
45 Ns = length(numssommets); % nombre de sommets
46
47 EE = zeros(Nbpt, Ns); % bloc rectangulaire (p, v1)
48 FF = zeros(Nbpt, Ns); % bloc rectangulaire (p, v2)
49
50 % On extrait de EEtmp et FFtmp les colonnes correspondantes a des sommets
51 % (par ailleurs les autres colonnes sont a priori nulles)
52 EE = EEtmp(:,numssommets);
53 FF = FFtmp(:,numssommets);
54
55 % Matrice elements finis par blocs
56 % -----
57 AA = sparse(2*Nbpt + Ns, 2*Nbpt + Ns);
58 %A_11
59 AA(1:Nbpt, 1:Nbpt) = nu*KK;
60 %A_22
61 AA((Nbpt+1):(2*Nbpt), (Nbpt+1):(2*Nbpt)) = nu*KK;
62 %A_13
63 AA(1:Nbpt, (2*Nbpt+1):(2*Nbpt + Ns)) = EE;
64 %A_23
65 AA((Nbpt+1):(2*Nbpt), (2*Nbpt+1):(2*Nbpt + Ns)) = FF;
66 %A_31
67 AA((2*Nbpt+1):(2*Nbpt + Ns), 1:Nbpt) = EE.';
68 %A_32
69 AA((2*Nbpt+1):(2*Nbpt + Ns), (Nbpt+1):(2*Nbpt)) = FF.';
70 % Autres composantes de AA sont nulle

```

Nous avons tout d'abord assemblé s'en se soucier des milieux d'arêtes mais avec dans des matrices temporaires pour les matrices $\mathbb{E}$ et $\mathbb{F}$. Ensuite, la liste des sommets a été extraite via le tableau `isavertex`, et les matrices des blocs associés à la pression, $\mathbb{E}\mathbb{E}$ et $\mathbb{F}\mathbb{F}$, ont été mises à jour pour ne conserver que les colonnes correspondantes aux sommets dans les matrices temporaires $\mathbb{E}\mathbb{E}\mathbb{tmp}$ et $\mathbb{F}\mathbb{F}\mathbb{tmp}$. Ensuite, la matrice $\mathbb{A}\mathbb{A}$ a été formée de manière creuse puis remplie pour ses 6 composantes non nulles.

4.4 Imposition des conditions par pseudo-élimination

Comme on peut le remarquer, nos matrices ne prennent pas en compte les conditions aux bords. Pour cela, nous allons utiliser la technique de pseudo-élimination pour imposer les conditions de bord.

Pour cela, nous avons élaboré la routine `elimine_stokes.m` :


```

1 function [tilde_AA, tilde_LL] = elimine_stokes(AA, LL, Refneu, Coorneu, probleme)
2
3 [nombre_points, ~] = size(Refneu);
4
5 % On initialise les matrices tilde_AA et tilde_LL qui contiendront les matrices
   modifi es
6 tilde_AA = AA;
7 tilde_LL = LL;
8
9 % Boucle sur chaque point pour traiter les conditions aux limites
10 for idx = 1:nombre_points
11     x=Coorneu(idx, 1);
12     y=Coorneu(idx, 2);
13
14     % Si refneu gal 1, cela signifie qu'on est sur une condition
15     % de Dirichlet, o les vitesses doivent tre fix es (u = g).
16     if Refneu(idx) == 1
17         % Annulation des lignes et colonnes de la matrice AA correspondantes
18         % au point
19         % pour que la vitesse soit impos e.
20         tilde_AA(idx, :) = 0;
21         tilde_AA(idx + nombre_points, :) = 0;
22
23         % Pour que la vitesse reste fix e aux valeurs (u1 et u2), on met 1
24         % sur les termes diagonaux correspondants (cf amphi 5) (afin
25         % d'avoir u1=g1, ...)
26         tilde_AA(idx, idx) = 1;
27         tilde_AA(idx + nombre_points, idx + nombre_points) = 1;
28
29         % Application des valeurs des conditions aux limites de Dirichlet
30         % pour les
31         % vitesses u1 et u2 (en fonction des coordonn es du point).
32         tilde_LL(idx) = g1(x,y,probleme); % Pour u1
33         tilde_LL(idx + nombre_points) = g2(x,y); % Pour u2
34     end
35
36     % Si refneu gal 2, cela signifie qu'on est sur une condition
37     % de Dirichlet
38     if Refneu(idx) == 2
39         % Annulation des lignes et colonnes de la matrice AA associ es ce
40         % point,
41         % car ces points n'ont pas d'inconnue associ e, seulement une
42         % condition de Dirichlet.
43         tilde_AA(idx, :) = 0;
44         tilde_AA(:, idx) = 0;
45         tilde_AA(idx + nombre_points, :) = 0;
46         tilde_AA(:, idx + nombre_points) = 0;
47
48         % Pour que la vitesse reste fix e aux valeurs (u1 et u2), on met 1
49         % sur les termes diagonaux correspondants (cf amphi 5) (afin
50         % d'imposer la condition la limite)
51         tilde_AA(idx, idx) = 1;
52         tilde_AA(idx + nombre_points, idx + nombre_points) = 1;
53
54         % On met z ro les valeurs du vecteur LL associ es ce point,
55         % pour des conditions de Dirichlet homog nes
56         tilde_LL(idx) = 0; % Pour u1
57         tilde_LL(idx + nombre_points) = 0; % Pour u2
58     end
59 end

```

Nous prenons en entrée la matrice du système linéaire "AA", le vecteur des termes sources "LL", ainsi que deux paramètres décrivant les conditions aux limites : "Refneu", qui identifie les nœuds du bord et précise les conditions appliquées à chaque nœud, et "Coorneu", qui contient les coordonnées des points du maillage. Nous modifions la matrice "AA" et le vecteur "LL" en annulant les lignes et colonnes des nœuds où des conditions de Dirichlet ou de Neumann sont imposées. Ensuite, nous appliquons les valeurs appropriées, en utilisant les fonctions "g1" et "g2" pour les conditions de Dirichlet, ou ajustons les termes sources pour les conditions de Neumann. En sortie, nous retournons la matrice modifiée "tilde_AA" et le vecteur "tilde_LL", qui tiennent compte des conditions aux limites appliquées sur les nœuds.

Les deux fonctions, 'g1' et 'g2', sont utilisées pour définir les conditions de Dirichlet sur les composantes de vitesse dans le cadre de notre problème de Stokes. Pour cela, nous créons deux routines, comme montrées ci-dessous :

```

1 function res = g1(x, y)
2 if x == 0
3     res = y * (2 - y);
4 else
5     res = 0;
6 end
7 end

```

```

1 function res = g2(x,y)
2 res = 0;
3 end

```

La fonction "g1(x, y)" calcule la condition de Dirichlet pour la composante de vitesse u_1 en fonction de la coordonnée y lorsque $x = 0$, renvoyant la valeur $y \cdot (2 - y)$, et 0 dans les autres cas. De son côté, la fonction "g2(x, y)" calcule la condition de Dirichlet pour la composante u_2 , mais cette condition est toujours nulle, renvoyant 0 quelle que soit la valeur des coordonnées x et y . Nous utilisons ainsi ces deux fonctions pour imposer des conditions spécifiques aux vitesses dans la résolution des équations aux dérivées partielles de Stokes. Nous avons déjà imposé des valeurs à la limite afin de vérifier notre code dans un second temps via une solution connue.

4.5 Vérification

Nous avons fini notre code pour résoudre l'équation de Stokes par les éléments finis. Nous aimerons vérifier qu'il approche correctement la solution, pour cela, nous testons sur notre code sur une solution connue. Pour cela, on résout le problème (9) avec le domaine rectangulaire $\Omega =]0, 3[\times]0, 2[$ et une viscosité $\nu = 1$. Pour les conditions au bord, nous avons :

- des conditions de Dirichlet homogènes (c'est-à-dire $(g_1, g_2) = (0, 0)$) sur les deux côtés horizontaux du rectangle ;
- des conditions de Dirichlet non homogènes avec $(g_1, g_2) = ((2 - y)y, 0)$ sur le côté gauche du rectangle, pour imposer un profil d'écoulement entrant parabolique ;
- des conditions de Neumann homogènes comme dans (7) en sortie, sur le côté vertical à droite du rectangle.

Nous avons imposé ces conditions limites via les routines `elimine_stokes.m`, `g1.m` et `g2.m`. La solution exacte est $(u_1, u_2) = ((2 - y)y, 0)$ et $p = -2(x - 3)$. Ces fonctions satisfont bien le problème (7). En effet, pour vérifier que les solutions exactes

$$(u_1, u_2) = ((2 - y)y, 0) \quad \text{et} \quad p = -2(x - 3)$$

satisfont le problème de Stokes (9), nous procédons comme suit :

4.5.1 Vérification des équations dans le domaine $\Omega =]0, 3[\times]0, 2[$

$$\begin{aligned} -\nu \Delta \mathbf{u} + \nabla p &= 0, \\ -\nabla \cdot \mathbf{u} &= 0. \end{aligned}$$

Pour $-\nu \Delta \mathbf{u} + \nabla p$:

- La dérivée seconde de $u_1(x, y) = (2 - y)y$ par rapport à x est nulle, :

$$\frac{\partial^2 u_1}{\partial x^2} = 0.$$

- La dérivée seconde de u_1 par rapport à y est égale à :

$$\frac{\partial^2 u_1}{\partial y^2} = \frac{\partial}{\partial y} (2 - 2y) = -2.$$

- $u_2(x, y) = 0$, donc :

$$\Delta u_2 = 0.$$

- Le gradient de la pression $p(x, y) = -2(x - 3)$ vaut :

$$\nabla p = \begin{bmatrix} \frac{\partial p}{\partial x} \\ \frac{\partial p}{\partial y} \end{bmatrix} = \begin{bmatrix} -2 \\ 0 \end{bmatrix}.$$

Ainsi, nous avons :

$$-\nu \Delta \mathbf{u} + \nabla p = -\nu \begin{bmatrix} \Delta u_1 \\ \Delta u_2 \end{bmatrix} + \begin{bmatrix} \frac{\partial p}{\partial x} \\ \frac{\partial p}{\partial y} \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix} + \begin{bmatrix} -2 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

L'équation $-\nu \Delta \mathbf{u} + \nabla p = 0$ est donc vérifiée.

Calcul de $-\nabla \cdot \mathbf{u}$:

Puisque $\frac{\partial u_1}{\partial x} = 0$ (car u_1 est indépendant de x) et $\frac{\partial u_2}{\partial y} = 0$ (car $u_2 = 0$), nous obtenons :

$$\nabla \cdot \mathbf{u} = 0.$$

L'équation $-\nabla \cdot \mathbf{u} = 0$ est donc vérifiée.

4.5.2 Vérification des conditions aux bords

- Sur Γ_D (bords gauche et haut) :

Sur le côté gauche ($x = 0$, $y \in]0, 2[$), on impose $\mathbf{u} = \mathbf{g}$ avec :

$$\mathbf{g} = ((2 - y)y, 0).$$

En utilisant les solutions exactes $u_1 = (2 - y)y$ et $u_2 = 0$, il est évident que cette condition est vérifiée.

Sur le côté haut ($y = 2$), $\mathbf{u} = \mathbf{0}$. En remplaçant $y = 2$ dans $u_1 = (2 - y)y$, nous obtenons $u_1 = 0$ et $u_2 = 0$. La condition est donc vérifiée. Sur le côté bas, ($y = 0$), la condition est aussi vérifiée.

- Sur Γ_N (bord droit) :

Sur le bord droit ($x = 3$), on impose une condition de Neumann homogène :

$$\nu \frac{\partial \mathbf{u}}{\partial \mathbf{n}} - p \mathbf{n} = 0.$$

Le vecteur normal est $\mathbf{n} = [1, 0]^T$. Calculons les contributions :

$$\nu \frac{\partial \mathbf{u}}{\partial \mathbf{n}} = \nu \begin{bmatrix} \frac{\partial u_1}{\partial x} \\ \frac{\partial u_2}{\partial x} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad -p \mathbf{n} = -(-2(3 - 3)) \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Ainsi :

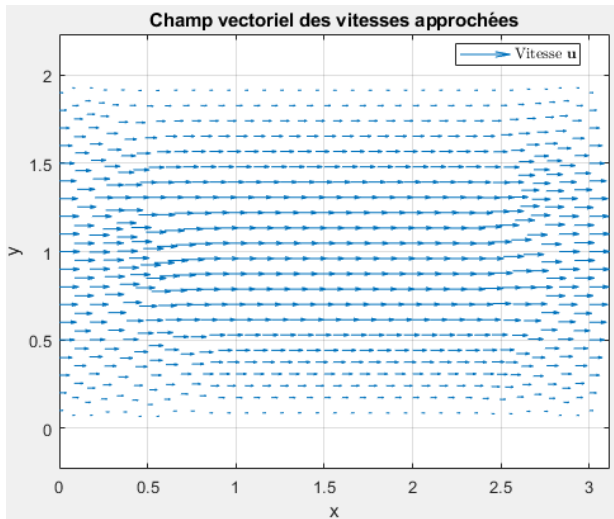
$$\nu \frac{\partial \mathbf{u}}{\partial \mathbf{n}} - p \mathbf{n} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

La condition de Neumann homogène est donc vérifiée.

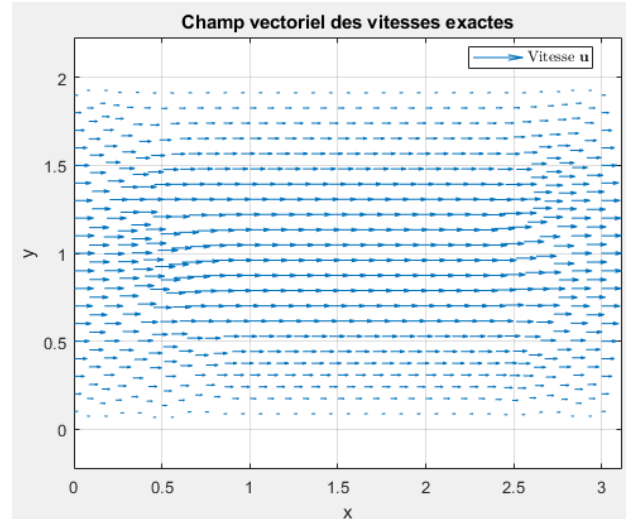
En conclusion, les solutions exactes $(u_1, u_2) = ((2 - y)y, 0)$ et $p = -2(x - 3)$ vérifient bien le problème de Stokes (9) et les conditions aux bords.

4.5.3 Solution approchée

Afin d'avoir une lecture plus simple des vecteurs vitesses, nous affichons notre solution approchée et exacte via des flèches dans notre code. Nous obtenons ainsi ces deux graphiques :



(a) Vitesse approchée

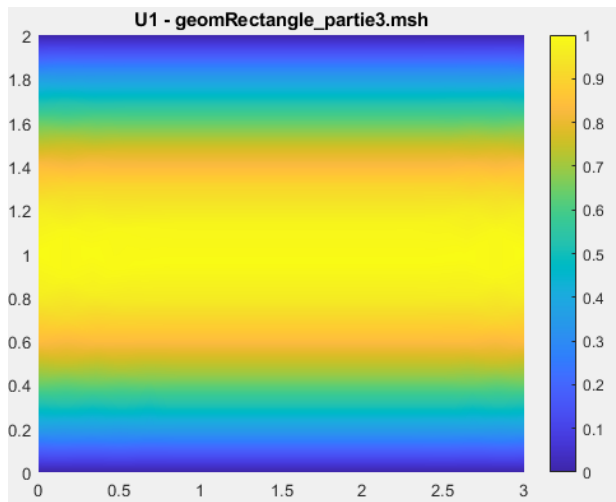
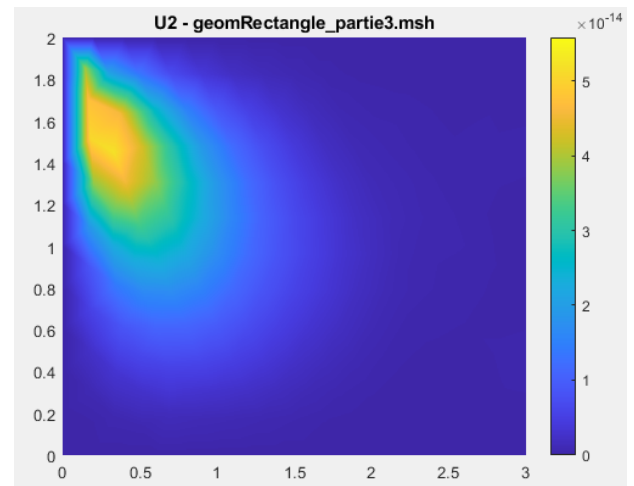
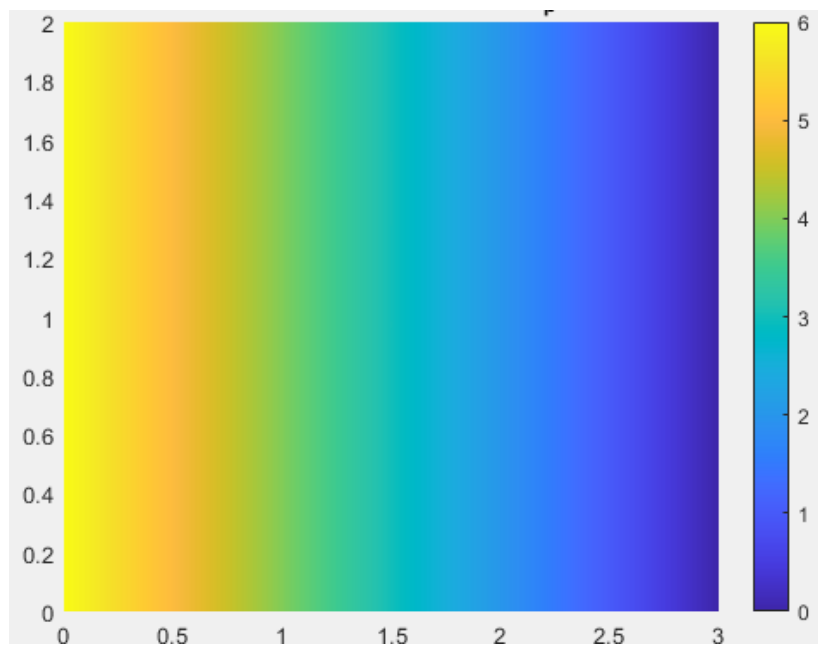


(b) Vitesse exacte

FIGURE 18 – Comparaison de la vitesse approchée et exacte

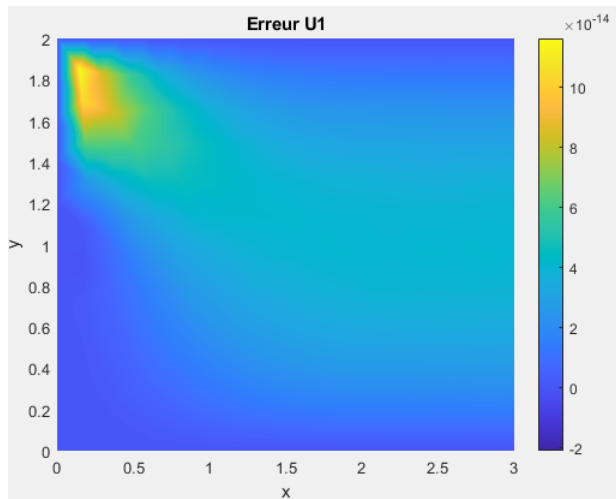
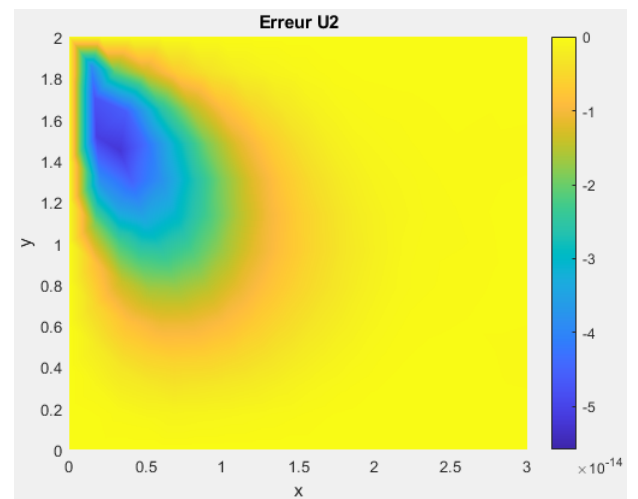
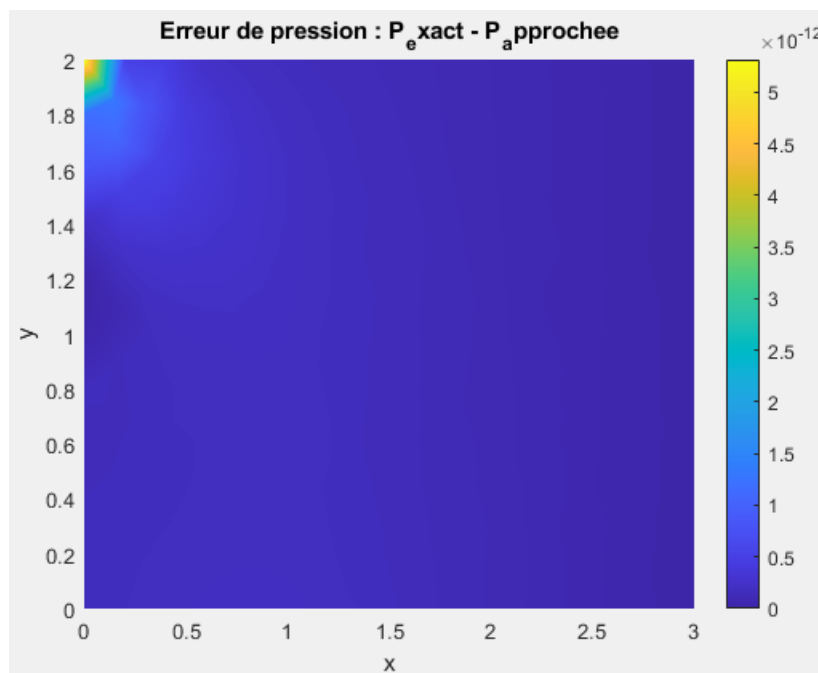
Dans un premier temps, nous remarquons que les solutions sont a priori similaires. En traçant les solutions approchées u_1 , u_2 et p avec un dégradé de couleur (cf 19), on s'aperçoit faiblement que u_1 ne varie qu'en fonction de y , u_2 est presque nulle (de l'ordre de 10^{-14}), et la pression ne dépend que de x comme attendu. Nous remarquons pour u_2 qu'une asymétrie (la tâche jaune) est probablement due à une asymétrie dans le maillage ou à des erreurs numériques mineures amplifiées par l'échelle de visualisation (valeurs très faibles, ordre de 10^{-14}).

Pour s'apercevoir mieux des erreurs commises, nous avons affiché dans 3 graphiques différents les différences $u_1 - u_{1h}$, $u_2 - u_{2h}$ et $p - p_h$ à la figure 20

(a) Erreur sur la vitesse horizontale u_1 (b) Erreur sur la vitesse verticale u_2 

(c) Erreur sur la pression de Stokes dégradée

FIGURE 19 – Représentation des différentes erreurs des grandeurs du problème de Stokes.

(a) Erreur sur la vitesse horizontale u_1 (b) Erreur sur la vitesse verticale u_2 

(c) Erreur sur la pression de Stokes dégradée

FIGURE 20 – Représentation des différentes erreurs des grandeurs du problème de Stokes.

Nous remarquons qu'une erreur plus élevée dans le coin en haut à gauche du rectangle. Cependant, elles sont quasi-nulles car de l'ordre de 10^{-14} et 10^{-12} . L'asymétrie des erreurs dans certaines régions pourrait indiquer des imperfections dans le maillage ou des conditions aux limites qui ne sont pas entièrement symétriques. Aussi, les erreurs observées peuvent être dues à des erreurs d'approximation numériques. Nous pourrions procéder à une évaluation de la convergence de la méthode en changeant la taille du maillage pour mieux comprendre comment ces erreurs évoluent lorsque l'on affine la solution.

4.6 Cas d'un domaine avec une marche

Nous allons maintenant complexifier légèrement le domaine, afin d'en voir son influence sur les erreurs. Nous avons étudié l'écoulement d'un fluide dans un canal présentant une marche. Le domaine de calcul est défini par plusieurs segments de bord, comme suit :

- Le segment de bord situé à gauche, où $1 \leq y \leq 2$, est noté Γ_{left} .
- Le bord de droite est désigné par Γ_{right} .
- Le bord supérieur est noté Γ_{up} .

— Le reste du bord, situé en bas, est noté Γ_{down} .

Les conditions aux limites appliquées sur ces segments sont les suivantes :

- Sur Γ_{left} , un profil d'écoulement entrant parabolique est imposé. Ce profil présente une vitesse maximale unitaire et est nul aux points $(0, 1)$ et $(0, 2)$. Cette condition a nécessité la modification du fichier `g1.m`.
- Une vitesse nulle est imposée sur les bords Γ_{up} et Γ_{down} .
- Des conditions de Neumann homogènes sont appliquées en sortie sur Γ_{right} .

Ensuite, nous avons modifié le fichier `.geo` pour générer un maillage d'ordre 2 du domaine décrit. Pour cela, nous avons 4 "physical line" pour les 4 bords. Cela, nous a donné un maillage pour un pas de $h=0.1$ comme montré dans le figure 21

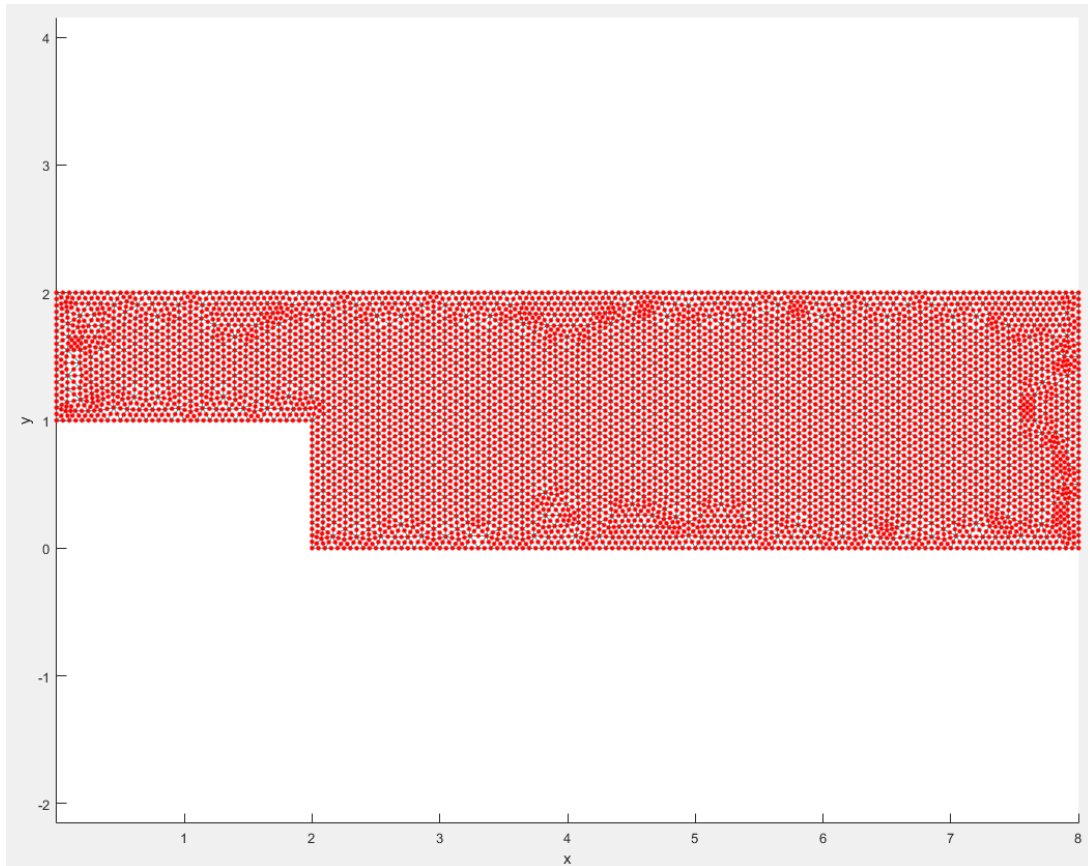


FIGURE 21 – Représentation du maillage pour le problème de la marche.

Un nouveau fichier principal, `principal_stokes_marche.m`, a été écrit afin de simuler l'écoulement dans le canal en utilisant les conditions aux limites définies précédemment.

Pour éliminer le doute sur une possible mauvaise identification des bords, nous allons implémenter dans la routine `elimine_stokes.m`, une figure 22a qui à chaque point du bord associe une couleur suivant son appartenance au type de bord. Voici un exemple d'ajout dans la routine pour le bord gauche inséré :

```
1 plot(Coorneu(idx, 1), Coorneu(idx, 2), 'yo', 'MarkerFaceColor', 'y');
```

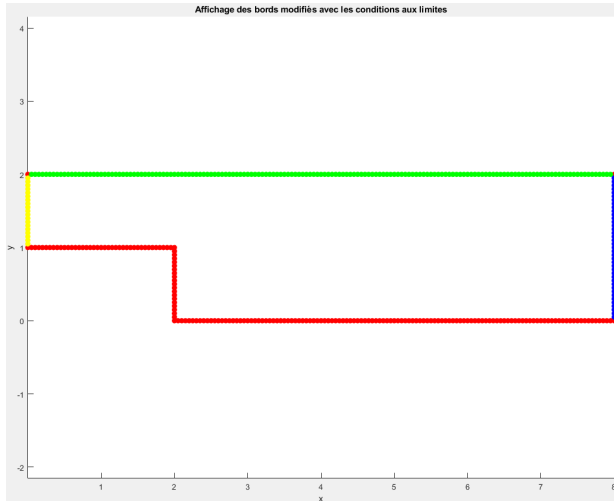
Listing 10 – Calcul de la matrice de masse élémentaire en P_2 Lagrange

Nous souhaitons aussi vérifier que la vitesse sur le bord gauche est bien le profil voulu. Nous avons mis dans la routine `g1.m`, pour le cas de la marche, ces lignes :

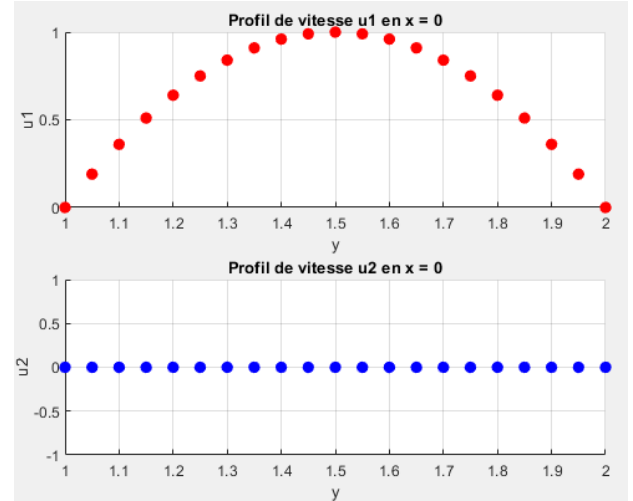
```
1 res = (y - 1) * (2 - y) / (0.25);
```

Listing 11 – Calcul de la matrice de masse élémentaire en P_2 Lagrange

Pour cela, nous traçons deux graphes pour les deux composantes de la vitesse.



(a) Vérification de l'appartenance des bords, jaune pour Γ_{left} , rouge pour Γ_{down} , bleu pour Γ_{right} , et vert pour Γ_{up}



(b) Vérification de la condition sur le bord gauche

Maintenant, pour analyser l'impact de la viscosité sur l'écoulement, nous avons effectué des simulations pour différentes valeurs de viscosité : $\nu = 1$, $\nu = 0.01$, et $\nu = 100$. Tout d'abord, théoriquement nous pouvons regarder les différents cas.

Rappel : Les équations de départ régissant l'écoulement s'écrivent :

$$\begin{cases} -\nu \Delta \mathbf{u} + \nabla p = 0 & \text{dans } \Omega, \\ \mathbf{u} = 0 & \text{dans } \Omega, \\ \mathbf{u} = \mathbf{g} & \text{sur } \Gamma_D, \\ \nu \frac{\partial \mathbf{u}}{\partial \mathbf{n}} - p \mathbf{n} = 0 & \text{sur } \Gamma_N, \end{cases} \quad (9)$$

a. Régime non visqueux ($\nu = 0.01$)

Lorsque $\nu \rightarrow 0$, le terme visqueux $-\nu \Delta \mathbf{u}$ devient négligeable. Les équations se réduisent à :

$$\nabla p = 0 \quad \text{dans } \Omega, \quad \mathbf{u} = \mathbf{g} \quad \text{sur } \Gamma_D.$$

La solution est alors déterminée par les conditions aux limites, et la pression reste constante dans Ω .

b. Régime très visqueux ($\nu \gg 1$)

Lorsque $\nu \gg 1$, le terme de pression devient moins influent, et l'équation est dominée par la diffusion visqueuse. Le système devient :

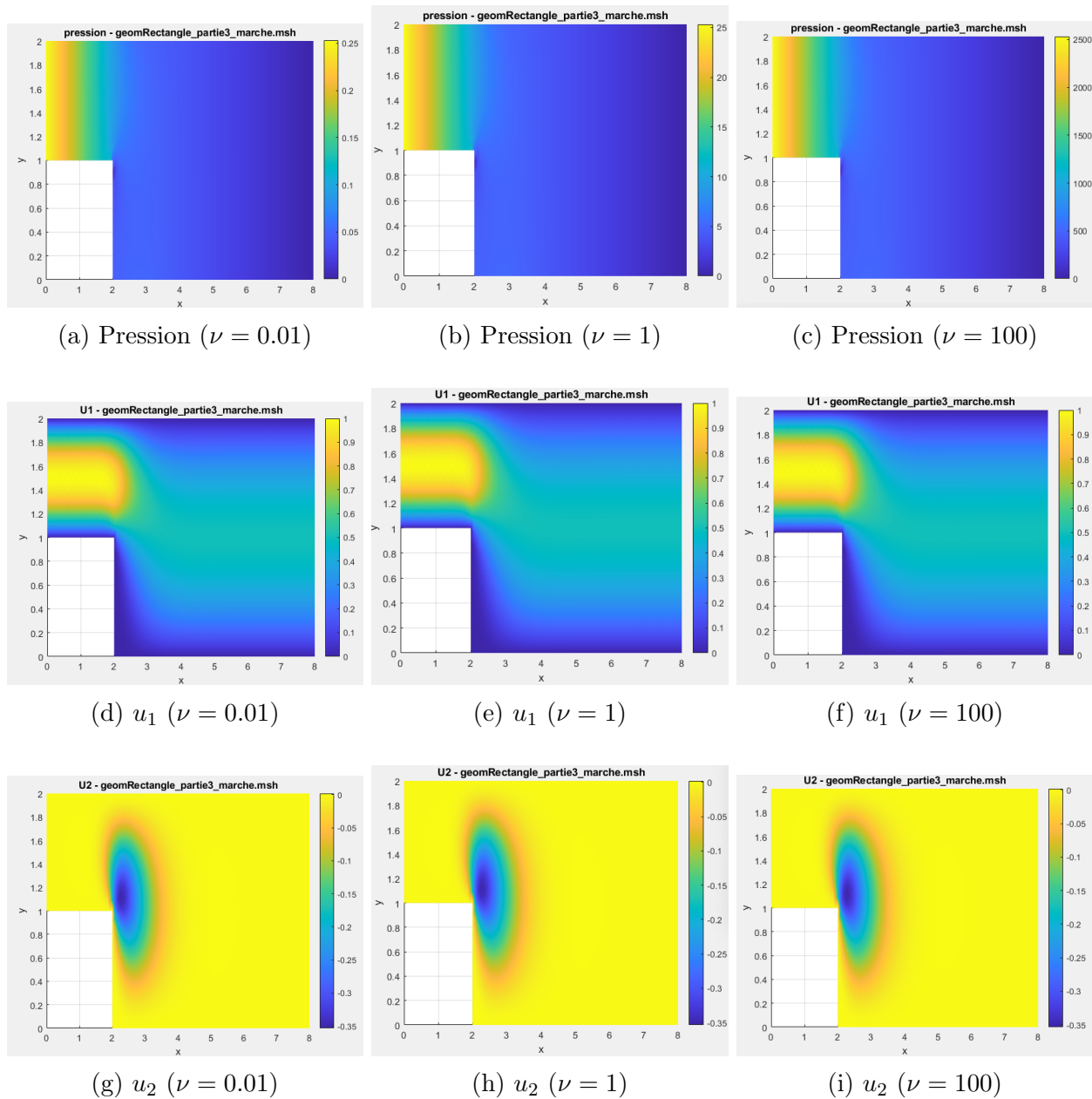
$$\Delta \mathbf{u} = 0 \quad \text{dans } \Omega, \quad \mathbf{u} = 0.$$

Ce régime correspond à un écoulement harmonique.

c. Régime intermédiaire (ν modéré)

Lorsque ν est modéré, il existe un équilibre entre les termes visqueux et la pression. Les équations complètes doivent être résolues, sans simplification supplémentaire.

Numériquement, voici ce que nous obtenons :

FIGURE 23 – Graphiques des variations de pression, u_1 , et u_2 pour différentes viscosités.

Nous remarquons que la vitesse est inchangée mais que la pression varie proportionnellement par rapport à ν . Cela s'explique par le fait que pour $\mathbf{u}$, nous avons pris des éléments P^2 , et donc le laplacien de $\mathbf{u}$ est une constante et donc p , qui est approchée par des éléments finis de P^1 est donc proportionnel à cette constante multipliée par ν à une autre constante près. Par exemple, " $p = \nu \times \text{constante1_de_u} + \text{constante2}$ ". La vitesse n'est pas affectée par ν du fait des conditions aux bords et de la nullité de la divergence de la vitesse dans le domaine. En faisant varier ν , nous avons pu valider partiellement notre code pour le cas de la marche (plus précisément, valider que les solutions approchées ont les bonnes dépendances par rapport à ν).

4.7 Intérêt des éléments finis P^1-P^2 par rapport aux éléments finis P^1-P^1

En prenant des éléments finis P^1-P^1 , on remarque que $\Delta u = 0$, donc d'après la première équation de Stokes. Le gradient de la pression est nul, donc la pression est constante. Et c'est ce que nous allons vérifier numériquement. Pour cela, nous reprenons le code `principal_stokes_marche.m`. Mais en modifiant les appels aux fonctions relatives à l'ordre 2 par celles relatives à l'ordre 1. Aussi, dans la

boucle for d'assemblage les indices limités à 6 par 3. Par exemple, dans la première sous-boucle, nous écrivons que

```

1 for i = 1:6 % Boucle sur les 6 degrés de liberté par triangle
2   I = Numtri(1, i); % Indice global du noeud i
3   for j = 1:6
4     J = Numtri(1, j); % Indice global du noeud j
5     MM(I, J) = MM(I, J) + Mel(i, j); % Assemblage masse
6     KK(I, J) = KK(I, J) + Kel(i, j); % Assemblage rigidité
7   end
8 end

```

Listing 12 – Assemblage corrigé pour P_2 Lagrange

Ainsi, nous obtenons les résultats suivants :

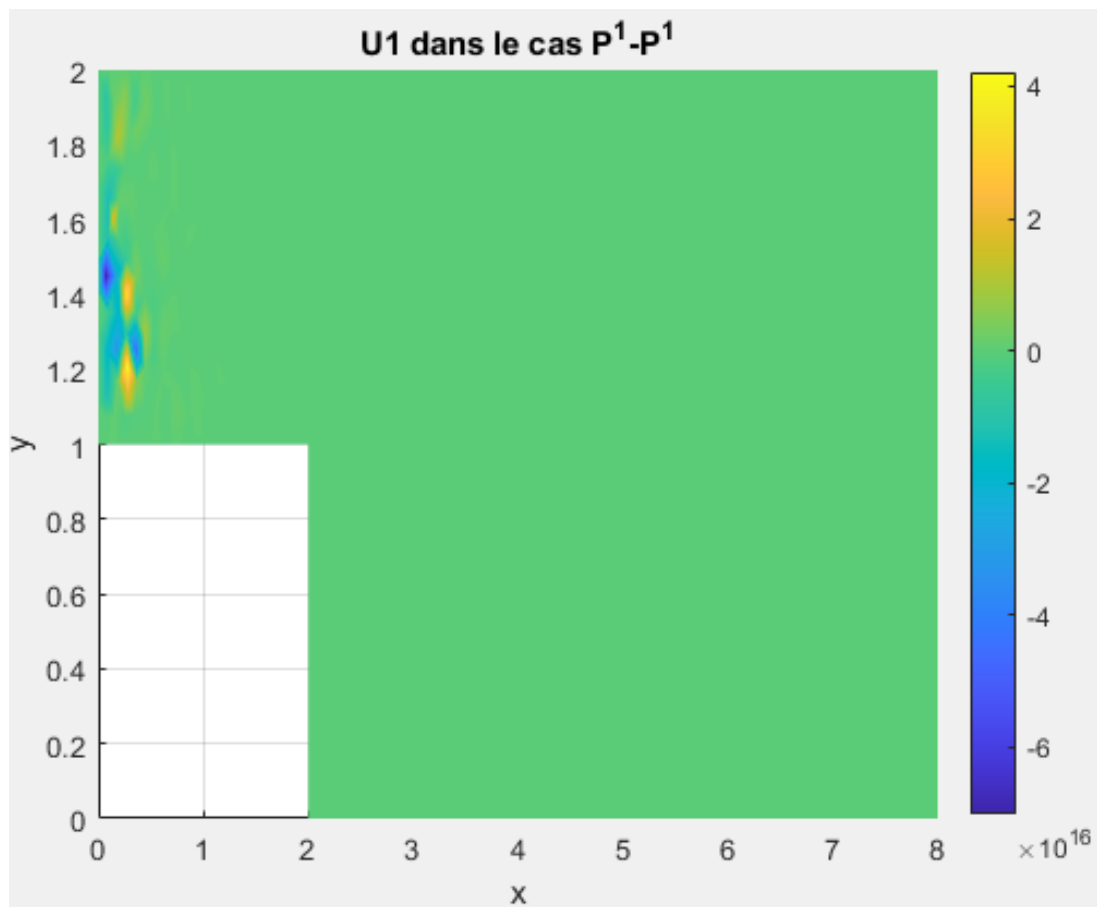
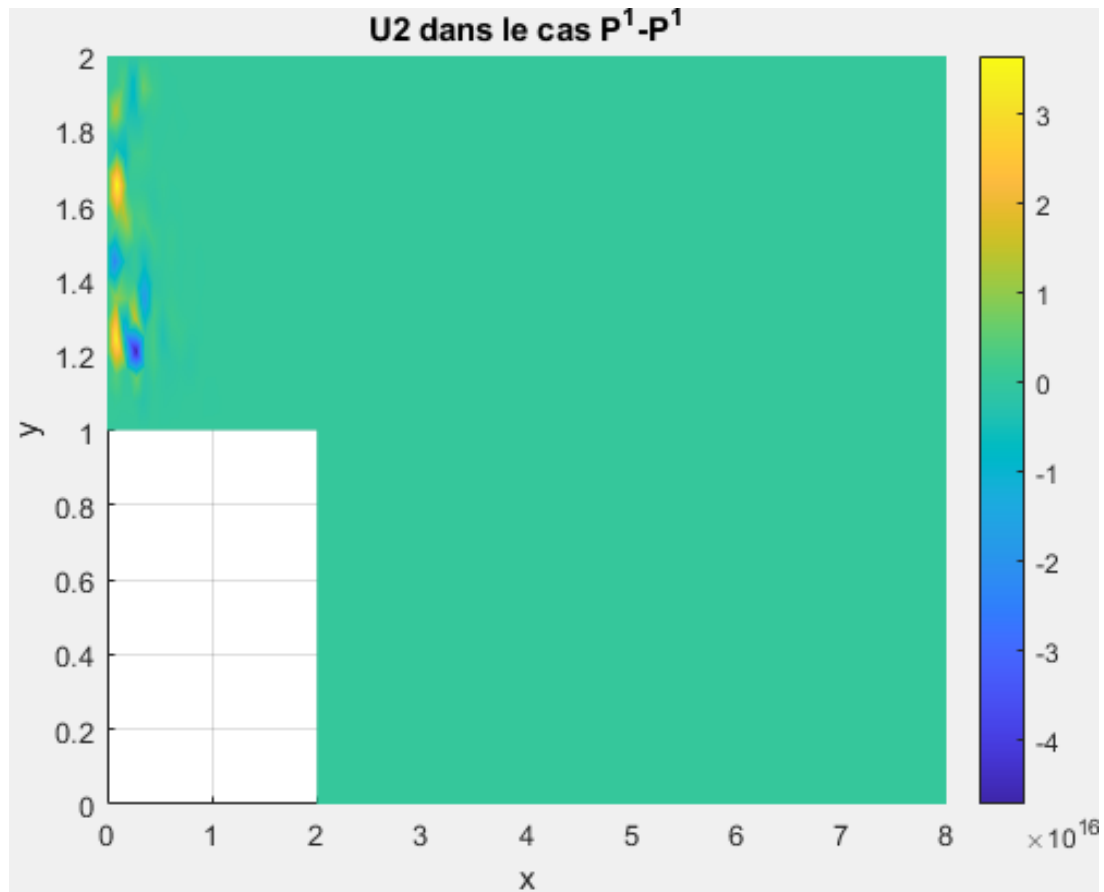
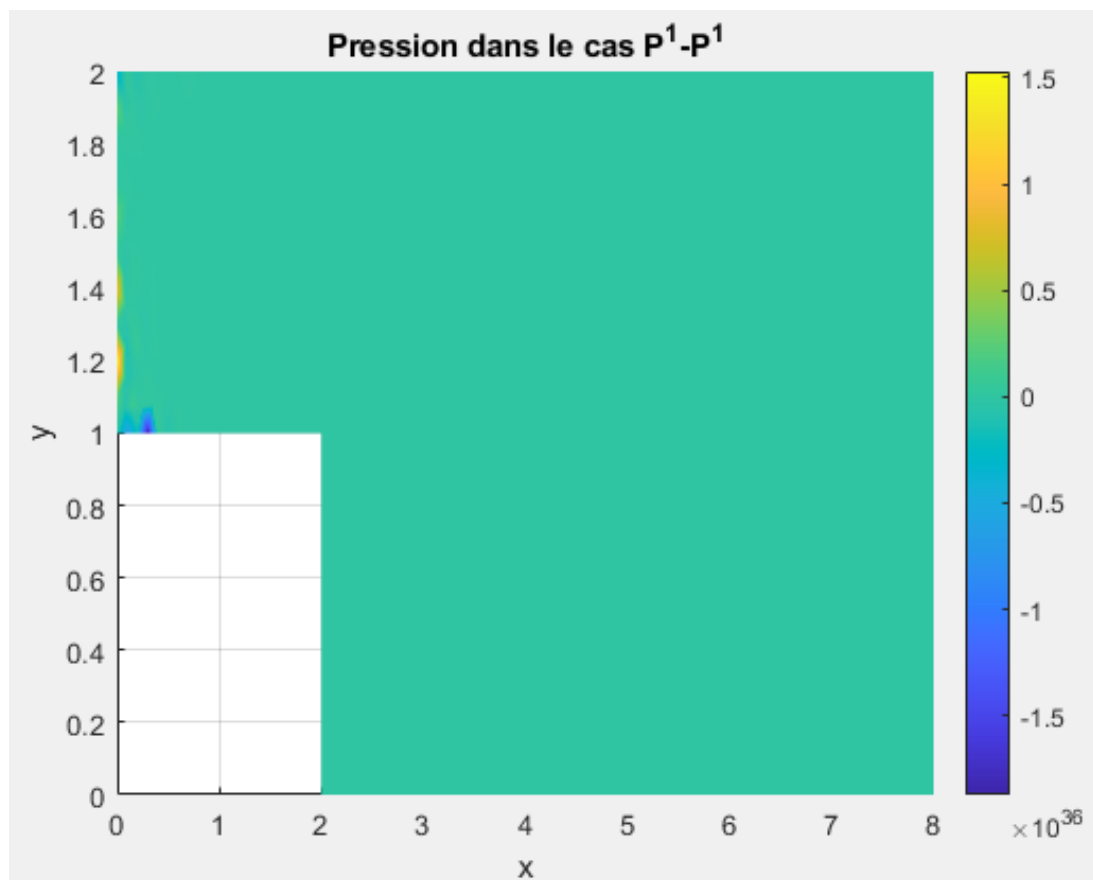


FIGURE 24 – Vitesse horizontale $U1$ dans le cas $P^1 - P^1$

FIGURE 25 – Vitesse verticale U1 dans le cas $P^1 - P^1$ FIGURE 26 – Pression dans le cas $P^1 - P^1$

Nous remarquons tout d'abord que la pression est nulle (exceptée sur certains endroits du bord

gauche et down). Ceci, vient du fait que le gradient de la pression (comme dit plus haut) est nul (car u_h est P^1 , donc son laplacien est nul). On ne peut pas imposer la condition sur le bord gauche, où g serait une parabole, car cela nécessiterait que u_h soit P^2 .

5 Conclusion

Dans ce rapport, nous avons étudié la simulation numérique de l'écoulement d'un fluide visqueux incompressible et stationnaire à faible nombre de Reynolds. Ce TP a consisté à mettre en œuvre les éléments finis P^2 pour résoudre des problèmes de Poisson avec des conditions aux limites de Neumann et de Dirichlet, avant de les appliquer aux équations de Stokes. Nos efforts se sont concentrés sur plusieurs aspects essentiels de la méthode. À savoir :

- **Discrétisation et assemblage** : Nous avons construit puis assemblé les matrices élémentaires et les seconds membres en utilisant des formules de quadrature pour les triangles de référence.
- **Validation des résultats** : Nous avons validé nos solutions numériques en les comparant à des solutions exactes dans des cas connus. Les erreurs, calculées en normes L^2 et H^1 , ont montré une convergence conforme aux attentes théoriques et ont permis d'évaluer l'impact du raffinement du maillage sur la précision. Pour le cas de l'équation de Stokes avec marche, nous avons juste regarder la sensibilité de la solution approchée à ν .
- **Simulation des équations de Stokes** : Nous avons simulé les équations de Stokes en discrétisant les composantes de vitesse avec des éléments finis P^2 et la pression avec des éléments finis P^1 (éléments de Taylor-Hood). Ces simulations ont permis de modéliser des écoulements, en tenant compte de différentes conditions aux limites et géométries.
- **Analyse des résultats** : Les représentations graphiques des solutions, notamment des champs de vitesse et des distributions de pression, ont permis une analyse qualitative et quantitative. Les différences entre solutions exactes et approchées ont également été effectuées.

Perspectives Ce TP nous a permis d'approfondir la théorie des éléments finis et de mettre en évidence leur efficacité pour la résolution de problèmes de mécanique des fluides notamment. Les éléments de Taylor-Hood ont montré leur intérêt pour les simulations effectuées, bien qu'il soit possible d'utiliser les éléments P^1 - P^1 pour d'autres cas. Enfin, l'étude de géométries plus complexes et de conditions aux limites variées offrirait des perspectives intéressantes pour des applications plus réalistes.

En conclusion, cette étude illustre la puissance des méthodes numériques pour résoudre des problèmes complexes tout en soulignant les défis liés à la précision et à la convergence des résultats.

Fin